



Conservatoire National des Arts et Métiers
Centre Paris

MÉMOIRE DE MASTER

présenté par : **Ahmed Atmane**

pour obtenir le grade de : **Master en Informatique**

Spécialité : **Réseaux & Objets Connectés (MR11606C)**

soutenance prévue : **septembre 2026**

InferRouter-LLM

**Routage dynamique d'intents réseau vers des LLM spécialisés
par estimation sémantique de complexité
et évaluation automatique de qualité**

Travail individuel — Continuité du projet RSX218

Abstract

Intent-Based Networking lets an operator state a goal in natural language rather than write configuration. Serving those intents with a large language model works, but it is expensive, and sending every intent to the strongest available model spends most of that budget on requests a cheaper model would have handled.

This thesis presents InferRouter-LLM, a router that selects, for each network intent, the cheapest model expected to meet a quality floor indexed on the intent’s criticality. It rests on three components: a semantic complexity estimator answering in 3.4 ms, a routing policy minimising cost under that floor, and a local LLM judge used for offline calibration.

Two findings stand out. The value of routing depends less on the cheap model’s absolute quality than on how predictable its weakness is: a uniformly capable cheap model removes the routing signal and makes informed routing lose against random selection, whereas a model whose quality decreases with complexity makes it work. Domain specialisation, in turn, only pays when domain routing is reliable, since applying a specialist outside its domain costs 3.6 times more than applying it correctly gains.

Beyond judge-based scoring, intents are applied as OpenFlow rules on an emulated network and verified in the data plane. Every check of that bench is validated by a negative and a positive control before any measurement is taken.

Keywords: intent-based networking, large language models, model routing, inference cost, semantic complexity estimation, LLM-as-a-judge, network automation, software-defined networking, quality of service, 5G.

Résumé

Le pilotage par intention permet à un opérateur d'exprimer un objectif en langage naturel au lieu d'écrire une configuration. Confier ces intentions à un grand modèle de langage fonctionne, mais coûte cher, et tout envoyer au modèle le plus puissant gaspille l'essentiel de la dépense sur des demandes qu'un modèle économique aurait traitées.

Ce mémoire présente InferRouter-LLM, un routeur qui choisit pour chaque intention réseau le modèle le moins coûteux censé atteindre un plancher de qualité indexé sur la criticité de l'intention. Il repose sur trois composants : un estimateur de complexité sémantique répondant en 3,4 ms, une politique de routage minimisant le coût sous ce plancher, et un juge local servant à la calibration hors ligne.

Deux résultats se dégagent. La valeur du routage tient moins à la qualité absolue du modèle économique qu'à la prévisibilité de sa faiblesse : un modèle économique uniformément bon supprime le signal de routage et fait perdre le routage informé face au tirage aléatoire, alors qu'un modèle dont la qualité décroît avec la complexité le rend exploitable. La spécialisation par domaine, de son côté, ne paie que si le routage par domaine est fiable, puisque appliquer un spécialiste hors de son domaine coûte 3,6 fois plus que le viser juste ne rapporte.

Au-delà de la notation par juge, les intentions sont appliquées en règles OpenFlow sur un réseau émulé et vérifiées dans le plan de données. Chaque vérification de ce banc est validée par un contrôle négatif et un contrôle positif avant toute mesure.

Mots-clés : réseaux pilotés par intention, grands modèles de langage, routage de modèles, coût d'inférence, estimation de complexité sémantique, évaluation par LLM juge, automatisation réseau, réseaux programmables, qualité de service, 5G.

Contents

Abstract	i
Résumé	ii
Chapters	
1 Introduction	1
1.1 Contexte et motivation	1
1.2 Problématique	2
1.3 Contributions	2
1.4 Plan du mémoire	3
2 État de l’art	4
2.1 Intent-Based Networking : fondations	4
2.2 LLM appliqués aux réseaux (panorama)	7
2.3 Routage et sélection dynamique de LLM	8
2.4 Évaluation automatique de qualité (LLM-as-a-Judge)	11
2.5 Synthèse et positionnement	13
3 Définition du problème	15
3.1 Modélisation (architecture) système	16
3.2 Challenges du système	18
3.3 Formulation du problème	19
4 InferRouter-LLM	24
4.1 Vue d’ensemble de l’architecture	24
4.2 Budget de latence par étage	24
4.3 Positionnement dans la boucle IBN et périmètre	26
4.4 Contribution 1 : estimateur de complexité sémantique	26
4.5 Contribution 2 : LLM cibles spécialisés par domaine réseau	27
4.6 Contribution 3 : LLM Juge (RocketEval)	27
4.7 Auto-calibration des seuils	28
5 Évaluation	29
5.1 Protocole expérimental	29
5.2 Estimateur de complexité sémantique	31

CONTENTS

5.3	Fiabilité du LLM Juge	33
5.4	Prémisse du routage et calibration	34
5.5	Benchmark comparatif des quatre stratégies	36
5.6	Le choix du léger : le meilleur modèle n'est pas le bon	38
5.7	Spécialisation par domaine	40
5.8	Validation en environnement réel	40
5.9	Problèmes rencontrés	42
5.10	Synthèse et limites	43
6	Conclusion	45
6.1	Synthèse	45
6.2	Apports scientifiques	45
6.3	Limites et perspectives	46
	Bibliography	48
	Acronyms	51
	Appendices	
A	Annexes techniques	53
A.1	Configuration du système	53
A.2	Mise en œuvre et rejeu	54
A.3	Exemples d'intents et réponses	54
A.4	Métriques classiques d'évaluation de texte	55

Chapter 1

Introduction

1.1 Contexte et motivation

L'adoption des grands modèles de langage (Large Language Models (LLMs)) en milieu industriel a franchi un seuil majeur sur la période 2023-2024. Selon le rapport AI Index 2025 de l'Institute for Human-Centered AI de l'Université de Stanford, 78 % des organisations déclarent recourir à l'intelligence artificielle dans au moins une fonction métier en 2024, contre 55 % un an plus tôt [1]. Dans le même intervalle, le coût d'inférence des LLM a chuté d'un facteur d'environ 280, grâce à la conjonction de modèles plus compacts, de techniques de quantification et d'optimisations d'exécution [1]. Cette double dynamique d'adoption massive et de baisse drastique des coûts unitaires déplace la frontière des usages : les LLM sortent du périmètre des applications conversationnelles généralistes pour s'établir comme briques opérationnelles de domaines verticaux exigeants, dont les télécommunications [2].

Le secteur réseau aborde simultanément une transition opérationnelle inédite. La densification des accès radio, la virtualisation des fonctions, le network slicing 5G et la multiplication des indicateurs de qualité à superviser confrontent les opérateurs à une complexité de gestion qui dépasse les capacités d'une administration purement humaine [3]. Le passage à la 6G accentue ce phénomène : la diversité attendue des services et la concurrence entre politiques de qualité de service rendent les conflits d'objectifs inévitables et imposent un raisonnement automatique au-delà des seuils classiques [4]. Ces constats motivent une nouvelle génération de systèmes d'automatisation, capables de raisonner sur l'intention de l'opérateur plutôt que sur des recettes de configuration manuelles.

L'Intent-Based Networking (Intent-Based Networking (IBN)) constitue la réponse paradigmatique à ce besoin. La RFC 9315 de l'Internet Research Task Force (IRTF) formalise un intent comme un ensemble d'objectifs et de résultats déclaratifs que le réseau doit satisfaire, sans spécifier la manière de les atteindre [5]. Le paradigme est également documenté en milieu industriel : Falkner et Apostolopoulos décrivent l'IBN comme l'architecture de référence pour les réseaux d'entreprise modernes et identifient la phase de traduction ("translation") de l'intent en configuration exécutable comme l'étape opérationnellement la plus fragile [6]. C'est précisément à cette phase que les LLM, dotés de capacités de compréhension du langage naturel, apparaissent comme des candidats techniques [7, 8]. L'usage des LLM pour la translation d'intents réseau ouvre néanmoins une série de questions opérationnelles non triviales. Un modèle unique ne couvre que médiocrement la diversité des domaines

1.2. PROBLÉMATIQUE

techniques sous-jacents (accès radio, cœur, sécurité, slicing), ce qui motive le recours à un ensemble de modèles spécialisés [2, 9]. L’inférence à l’échelle d’un opérateur engage des coûts significatifs qui justifient un arbitrage économique systématique [10]. La latence ajoutée par un appel LLM doit rester compatible avec les boucles d’assurance des réseaux. Enfin, l’évaluation de la qualité d’une réponse à un intent réseau ne peut reposer durablement sur une supervision humaine à grande échelle. Ces quatre tensions (spécialisation, coût, latence et qualité vérifiable) constituent le point de départ du présent travail.

1.2 Problématique

L’état de l’art du routage de LLM et de l’application des LLM aux réseaux montre que les axes de spécialisation, de routage multi-critère et d’évaluation automatique sont aujourd’hui traités de manière séparée. Les approches de routage économique [10] concentrent l’arbitrage sur le couple coût-qualité, sans contrainte de latence évaluée intent par intent. Les travaux sur les LLM appliqués aux réseaux [9, 2] démontrent la valeur d’une spécialisation par domaine, sans formaliser le mécanisme de sélection. Hong identifie d’ailleurs le routage et la spécialisation comme verrous ouverts pour le passage à l’échelle opérateur [11].

À notre connaissance, aucun travail ne combine simultanément (a) une spécialisation par domaine réseau, (b) un routage tri-critère sous contraintes dures de coût et de latence évaluées par intent et (c) une évaluation automatique de qualité à la fois fiable et économique (cf. §2.5.2).

Ce manque est problématique pour les opérateurs réseau. Les volumes d’intents traités quotidiennement, les engagements de service contractuels et les budgets d’inférence alloués par client interdisent une approche fondée sur un modèle unique ou sur une supervision humaine systématique. L’absence d’un cadre conjoint coût-latence-qualité produit en pratique des systèmes qui optimisent un critère au détriment des autres et qui restent dépendants d’une boucle d’évaluation humaine non passable à l’échelle.

Le présent mémoire s’articule autour de la question de recherche suivante : “comment router dynamiquement des intents réseau hétérogènes vers des LLM spécialisés en respectant des contraintes dures de coût et de latence par intent, sans humain dans la boucle pour évaluer la qualité ?” Cette question fixe simultanément la portée du travail (intents réseau, pas requêtes génériques), son cadre formel (contraintes dures évaluées par intent) et son exigence opérationnelle (autonomie de l’évaluation).

1.3 Contributions

Pour répondre à la question posée, le présent travail introduit InferRouter-LLM, un système de routage d’intents réseau articulé autour de trois contributions complémentaires.

1.3.0.0.1 Estimateur sémantique de complexité d’intents réseau. La première contribution est un estimateur de complexité calibré sur un corpus d’intents réseau généré par LLM (quatre domaines, trois niveaux de complexité), et qui répond en 3,4 ms, soit un coût négligeable devant l’inférence qu’il pilote. Le système retient les attributs de fond de l’énoncé (nombre d’entités, profondeur des contraintes, diversité des domaines), définis au chapitre 3 (§3.2) ; une variante fondée sur des représentations `sentence-transformers` atteint une exactitude supérieure et sert de référence, au prix d’une dépendance que le prototype n’assume pas (§5.2).

1.3.0.0.2 Routage à coût minimal sous plancher de qualité. La deuxième contribution formalise le routage comme la sélection, dans un ensemble admissible défini par intent, du modèle le moins coûteux dont la qualité attendue $\hat{q}_i(e)$ atteint un plancher $q_{\min}(e)$ indexé sur la criticité de l'intent, sous des bornes dures de coût $C_{\max}(e)$ et de latence $L_{\max}(e)$ (chapitre 3, éq. (3.5) et (3.6)).

Le sens de cet arbitrage est délibéré. Maximiser la qualité sous budget conduit à servir le modèle le plus fort partout où le budget le permet, ce qui annule le routage. Minimiser la dépense sous garantie de qualité laisse au contraire à l'opérateur un curseur explicite : il paie le modèle fort là où la criticité l'exige et économise ailleurs. À notre connaissance, cette formulation, avec un plancher indexé sur la criticité et des bornes évaluées par intent, est inédite pour le routage de LLM appliqué aux réseaux, verrou explicitement identifié par Hong [11].

1.3.0.0.3 LLM-Juge local avec auto-calibration. La troisième contribution intègre un LLM-Juge local (gemma2 via Ollama) pour évaluer automatiquement la qualité des réponses. Il applique la méthode RocketEval [12], fondée sur des checklists instanciées par intent, qui rapporte une corrélation de Spearman de l'ordre de 0,965 avec les préférences humaines pour un coût marginal quasi nul. L'exécution locale élimine la dépendance à une Application Programming Interface (API) cloud et calibre les seuils du routeur sans intervention humaine. Notre évaluation (chapitre 5) circonscrit cet usage : le juge est fiable pour départager des candidats nettement différents, donc pour le routage, mais pas comme oracle d'une qualité absolue fine.

Prises ensemble, ces contributions forment la couche de sélection de modèle de la phase de translation d'une boucle IBN : elles rendent économiquement soutenable l'insertion de LLM dans cette boucle, sans en modifier les garde-fous d'activation et d'assurance. Le positionnement dans la boucle et le périmètre retenu (l'actuation sur l'infrastructure reste hors champ) sont explicités au chapitre 4 (§4.3). Ces trois contributions sont décrites en détail au chapitre 4 (4) et évaluées expérimentalement au chapitre 5 (5).

1.4 Plan du mémoire

Le chapitre 2 (2) développe l'état de l'art : fondations de l'IBN, LLM appliqués aux réseaux, routage dynamique de LLM, évaluation LLM-as-a-Judge, puis positionnement. Le chapitre 3 (3) formalise le problème (notations, hypothèses, critères de complexité, métriques). Le chapitre 4 (4) présente l'architecture d'InferRouter-LLM et ses trois contributions. Le chapitre 5 (5) rapporte l'évaluation expérimentale (qualité, coût, latence, ablations). Le chapitre 6 (6) conclut sur les apports, les limites et les perspectives.

Chapter 2

État de l’art

2.1 Intent-Based Networking : fondations

2.1.1 Définition : intents, IBN et héritage autonome

L’objet manipulé par InferRouter-LLM est l’*intent réseau*. Nous en fixons le sens, par opposition aux notions voisines de politique (“policy”) et de modèle de service (“service model”).

2.1.1.0.1 Intent réseau. La RFC 9315 de l’IRTF définit un intent comme “un ensemble d’objectifs opérationnels qu’un réseau doit satisfaire et de résultats qu’il doit produire, exprimés de manière déclarative sans spécifier comment les atteindre” [5]. Cette définition implique deux propriétés distinctives. D’une part, l’intent est déclaratif : il énonce un résultat attendu (ce que le réseau doit accomplir) et non une procédure de configuration (comment y parvenir). D’autre part, il est formulé à un haut niveau d’abstraction, typiquement compréhensible par un opérateur humain sans connaissance des artefacts de configuration sous-jacents (commandes CLI, primitives YANG, manifestes NETCONF). À ce double titre, l’intent se distingue d’une politique classique, qui prescrit généralement un comportement *event-condition-action* explicite [5], ainsi que d’un modèle de service, qui décrit la structure d’un service offert mais ne spécifie pas nécessairement les objectifs opérationnels associés. Nous notons $e \in E$ pour désigner un intent appartenant à l’ensemble fini d’intents traités par le système, conformément à la formalisation du chapitre 3 (cf. §3.2).

2.1.1.0.2 Intent-Based Networking (IBN). L’IBN désigne le paradigme opérationnel qui prend l’intent comme primitive d’interaction entre l’opérateur et le réseau. Un Intent-Based Networking System (IBNS), selon la taxonomie de Leivadeas et Falkner [7], articule cinq composantes formant une boucle fermée d’automatisation :

1. *intent expression* : saisie de l’intent par l’opérateur, en langage naturel ou via une interface guidée ;
2. *intent translation* : transformation de l’intent déclaratif en une représentation intermédiaire formelle, exploitable par les couches de configuration ;

2.1. INTENT-BASED NETWORKING : FONDATIONS

3. *intent resolution* : détection et résolution des conflits entre intents concurrents ou avec l'état courant du réseau ;
4. *intent activation* : déploiement effectif de la configuration résultante sur les équipements (physiques ou virtualisés) ;
5. *intent assurance* : vérification continue que les résultats observés respectent l'intent initial, avec rétroaction corrective.

La RFC 9315 énonce aussi six principes opératoires (source unique de vérité, raffinement itératif, autonomie supervisée, apprentissage continu, exposition de capacités, abstraction) [5]. Le présent travail se concentre sur la phase de translation, identifiée comme étape critique [13, 8] : c'est là que la diversité lexicale des intents entre en collision avec la rigidité des langages de configuration cibles.

2.1.1.0.3 Héritage autonome et normalisation. L'IBN prolonge l'*autonomic networking* initié par IBM dans les années 2000, en tirant parti de l'apprentissage automatique pour rendre opérationnelle l'auto-gestion [7]. Plusieurs organismes en portent la normalisation : l'IRTF (RFC 9315 [5]), le Third Generation Partnership Project (3GPP) (gestion 5G/6G) et l'European Telecommunications Standards Institute (ETSI) via le groupe Zero-touch network and Service Management (ZSM) [14]. Les quatre surveys de référence [13, 15, 7, 8] convergent : l'IBN est mature conceptuellement, mais bute sur un même verrou, la traduction robuste d'intents en langage naturel vers des configurations exécutables. C'est ce verrou qui motive ce travail.

2.1.2 Domaines d'application : RAN, slices 5G, sécurité, cœur réseau

Les concrétisations de l'IBN varient selon le sous-système ciblé. Suivant la cartographie de Mehmood *et al.* [8] et de Leivadeas et Falkner [7], nous retenons quatre domaines, qui structureront l'ensemble $\mathcal{D}$ des spécialisations du routeur (chapitre 3).

2.1.2.0.1 Réseau d'accès radio (Radio Access Network (RAN)). Les intents RAN portent sur l'optimisation des paramètres radio (puissance, beamforming, Modulation and Coding Scheme (MCS)), le placement de cellules et l'équilibrage de charge. Le caractère quantifiable des objectifs (couverture, débit, interférence) en fait un domaine pionnier de l'IBN [13] ; le RAN Intelligent Controller (RIC) des architectures O-RAN offre un point d'ancrage concret via les *xApps* et *rApps* [8].

2.1.2.0.2 Tranches de réseau (slices) 5G. Un intent slice spécifie une classe de service (Ultra-Reliable Low-Latency Communications (uRLLC), enhanced Mobile BroadBand (eMBB), massive Machine Type Communications (mMTC)) assortie de garanties Service Level Agreement (SLA), traduite en provisionnement cohérent d'une Network Slice Instance (NSI) sur les segments d'accès, de transport et de cœur [15]. L'isolation entre tranches et les conflits inter-slices sont des défis bien adressés par la déclarativité [8].

2.1.2.0.3 Sécurité réseau. L'IBN appliqué à la sécurité (Intent-Based Security (IBSec)) porte sur les politiques d'isolation (Access Control List (ACL), segmentation), les pare-feu distribués et les Intrusion Detection System (IDS). Un objectif de protection s'exprime indépendamment de la topologie et se

re-traduit dynamiquement [7] ; sa traduction reste toutefois largement manuelle en industrie, faute de vérification formelle mature [8].

2.1.2.0.4 Cœur de réseau (core). Les intents cœur 5G configurent les fonctions Access and Mobility Management Function (AMF), Session Management Function (SMF), User Plane Function (UPF) et les politiques de Quality of Service (QoS) du Policy Control Function (PCF) [8]. La complexité tient à l’enchevêtrement des dépendances entre fonctions virtualisées : un intent peut exiger la reconfiguration coordonnée de plusieurs Network Function (NF) [15].

Ces quatre domaines fondent la taxonomie de l’estimateur de complexité (§3.2) : la difficulté d’un intent dépend de sa surface lexicale, mais aussi du domaine cible et du nombre de frontières inter-domaines franchies. La spécialisation des LLM cibles par domaine y prend racine.

2.1.3 Limites des approches IBN classiques

Malgré la maturité conceptuelle du paradigme et la convergence des standards, les réalisations opérationnelles d’IBN antérieures aux modèles de langage de grande taille restent confrontées à trois limites récurrentes, identifiées de manière convergente par les surveys [13, 15, 7, 8] et illustrées par les systèmes d’époque.

2.1.3.0.1 Couverture lexicale limitée. Les approches antérieures reposent sur du Natural Language Processing (NLP) à base de règles, d’ontologies (Web Ontology Language (OWL)) et de grammaires spécialisées. Le système LUMI [16] en est l’archetype : Named Entity Recognition (NER) et règles de transformation vers le langage formel NILE, avec feedback humain pour les ambiguïtés. Évalué sur 50 réseaux campus, il généralise mal aux paraphrases hors base d’apprentissage. Les avancées en NLU sont ainsi la “brique manquante” pour passer de l’IBN théorique à l’IBN opérationnel [15].

2.1.3.0.2 Fragilité face aux intents nouveaux. Tout nouveau type d’intent exige l’ajout manuel d’un patron, d’une entrée ontologique ou d’une règle dans le Domain-Specific Language (DSL) cible. Cette charge croît avec la couverture visée, frein industriel majeur [7], formalisé comme un défi de “scalabilité sémantique” par Mehmood *et al.* [8].

2.1.3.0.3 Absence d’évaluation automatique. La vérification que la traduction intent $\rightarrow$ configuration est correcte repose sur une validation humaine ou des tests coûteux [13]. Aucun mécanisme générique n’émet un jugement automatique à partir du seul couple (intent, configuration), et les métriques lexicales (BLEU, ROUGE) sont inadaptées à des configurations exigeant une correction fonctionnelle. Cette carence est un frein majeur au déploiement closed-loop [13, 14].

Ces trois limites (couverture lexicale, fragilité face à la nouveauté et carence d’évaluation automatique) définissent le cahier des charges implicite que les approches récentes fondées sur les LLM ambitionnent de satisfaire. La section suivante (§2.2) examine précisément comment les LLM ont été mobilisés pour lever ces verrous, ouvrant la voie à la problématique du routage entre modèles traitée en §2.3.

2.2 LLM appliqués aux réseaux (panorama)

Depuis 2023, la communauté réseau transpose les capacités des LLM généralistes aux tâches laissées ouvertes par l'IBN classique (§2.1.3). Deux surveys [17, 18] partitionnent ce champ en trois familles : génération de configurations, diagnostic et analyse de journaux, traitement d'intents en langage naturel. Nous en retenons trois constats qui motivent la problématique du routage (§2.3) : l'hétérogénéité des tâches, la sensibilité des modèles généralistes au domaine [18, 19], et l'usage quasi-systématique d'un modèle cloud unique sans arbitrage coût-qualité-latence [9, 20].

2.2.1 Génération de configurations

La première famille explore la production de configurations d'équipements (Cisco IOS, Juniper Junos, NETCONF) à partir d'énoncés de haut niveau. C'est le banc d'essai le plus structuré du panorama, car la tâche se prête à une évaluation par vérificateurs symboliques.

2.2.1.0.1 NetConfEval : un banc d'essai de référence. L'étude conduite par Wang *et al.* [21], publiée aux *Proceedings of ACM on Networking* (CoNEXT 2024) et récompensée par le *Applied Networking Research Prize* de l'IRTF, propose le benchmark NETCONFVAL, qui évalue plusieurs LLM (GPT-3.5, GPT-4, Code Llama) sur quatre tâches graduées de difficulté croissante : (i) traduction d'exigences exprimées en langage naturel vers une spécification formelle, (ii) génération d'appels API et de fonctions, (iii) implémentation d'algorithmes de routage, et (iv) génération de configurations bas-niveau pour des protocoles tels que BGP ou OSPF [21].

L'apport méthodologique majeur de NetConfEval est de montrer que la qualité des sorties LLM dépend non linéairement de la complexité syntaxique des artefacts attendus : les modèles réussissent mieux la traduction d'exigences vers une représentation intermédiaire formelle que la génération directe de configurations bas-niveau, dont la fragilité augmente avec le nombre de fonctionnalités couplées. Ce constat fonde la justification d'un estimateur de complexité, brique centrale du présent travail, et fixe le vocabulaire d'évaluation que nous reprenons au chapitre 5.

Trois travaux complètent ce constat. NETLLM [19] adapte un même LLM à toutes les tâches réseau par une procédure inspirée de Low-Rank Adaptation (LoRA) : c'est la position opposée au routage vers des modèles distincts, contraste exploité en §2.5. Mondal *et al.* [22] montrent que GPT-4 seul produit des configurations affectées d'erreurs structurelles et proposent un couplage à un vérificateur symbolique externe ; ce principe hybride LLM + validateur fonde notre LLM Juge. GENET [23] ajoute la lecture multimodale d'une topologie et la récupération documentaire (Retrieval-Augmented Generation (RAG)), piste hors périmètre renvoyée aux perspectives (chapitre 6). Ces travaux convergent : la génération reste sensible aux erreurs sémantiques dès le seuil de fonctionnalités couplées identifié par NetConfEval [21], ce qui justifie une étape diagnostique préliminaire.

2.2.2 Diagnostic et analyse de logs

La deuxième famille exploite la capacité des LLM à interpréter de grands volumes de texte semi-structuré pour assister l'opérateur dans le diagnostic d'incidents et la corrélation d'événements.

HERMES [24] orchestre plusieurs agents LLM par *blueprints* pour construire des jumeaux numériques réseau, et CHATNET [9] découpe le diagnostic en quatre rôles instanciés par un même GPT-4. Les deux illustrent le passage du LLM unique à une chaîne d'agents, voisine de notre routage vers experts,

2.3. ROUTAGE ET SÉLECTION DYNAMIQUE DE LLM

mais orientée rôles fonctionnels plutôt que domaines réseau, et sans arbitrage coût/latence/qualité. Le survey de Zhou *et al.* [18] identifie précisément les enjeux que ces systèmes laissent ouverts : confidentialité, latence sur longs contextes et coût de l’invocation répétée d’un modèle cloud unique. Ce dernier enjeu motive la problématique du routage (§2.3).

2.2.3 Traitement d’intents en langage naturel

La troisième famille cible explicitement la phase de translation d’un intent en langage naturel vers une représentation exploitable par les couches de configuration ; c’est celle dans laquelle InferRouter-LLM s’inscrit le plus directement.

Manias *et al.* [25] extraient à DRCN 2024 des intents 5G structurés (JavaScript Object Notation (JSON)) et montrent que le conditionnement par un contexte de domaine explicite (cœur, RAN, slice) améliore l’extraction, premier argument empirique pour un découpage par domaine.

2.2.3.0.1 Routage sémantique pour la gestion intent-driven 5G. La même équipe prolonge ce travail avec une contribution particulièrement proche du présent mémoire [20] : un *semantic router* encode l’intent via SENTENCE-BERT ou MINILM pour sélectionner le sous-système cible, améliorant précision et coût face au prompting pur. La différence est structurante : chez Manias *et al.*, le routage sémantique désigne le composant réseau visé (fonction du cœur 5G) ; chez InferRouter-LLM, il désigne le LLM cible adapté à la complexité de l’intent. Les deux approches sont complémentaires, et cette distinction est reprise en §2.5.

Deux travaux complètent le tableau. Mekrache et Ksentini [26] traduisent des intents vers des Network Service Descriptor (NSD) avec boucle *Human Feedback* (OSS-GPT, Eurecom), en aval d’un éventuel routeur. INTA [27] traduit des configurations inter-vendeur (Cisco → Juniper) via une représentation intermédiaire d’intent, avec une exactitude syntaxique de 98,15 %, confirmant la viabilité de l’intent comme représentation commune.

Ces quatre contributions partagent un schéma commun : toutes mobilisent un unique LLM en aval, le plus souvent un modèle cloud propriétaire (GPT-3.5, GPT-4) appelé via API, sans mécanisme explicite d’arbitrage entre plusieurs modèles candidats. Lorsque l’orientation sémantique existe (Manias *et al.* [20]), elle vise un composant opérationnel et non un outil de raisonnement. Cette absence d’un arbitrage tri-critère coût-qualité-latence à la couche “outil de raisonnement” constitue précisément le *gap* adressé par le présent mémoire ; la section suivante en propose une formalisation à partir de la littérature du routage entre LLM.

2.3 Routage et sélection dynamique de LLM

Les systèmes examinés (§2.2) invoquent quasi-systématiquement un modèle cloud unique, quelle que soit la complexité de la requête [9, 24, 18]. Or, face à un pool hétérogène aux compromis coût/qualité/latence distincts, il existe pour chaque requête un modèle plus efficient qu’un choix uniforme [10, 28]. Le routage de LLM consiste à apprendre, pour une requête donnée, le modèle qui maximise une utilité sous contraintes.

La littérature s’organise en deux familles complémentaires. Les approches *model-centric* construisent une représentation des LLM cibles (classement Elo, reward models distillés, embeddings appris) et y projettent la requête. Les approches *query-centric* font l’inverse : elles caractérisent la requête

2.3. ROUTAGE ET SÉLECTION DYNAMIQUE DE LLM

(difficulté estimée, embedding, auto-évaluation) avant de la mapper à une politique de sélection. Des approches hybrides combinent les deux, typiquement les cascades du moins cher au plus cher avec validation intermédiaire.

2.3.1 Approches centrées modèle (model-centric)

Les approches model-centric postulent que le prédicteur de qualité le plus informatif n’est pas la requête elle-même, mais la cartographie relative des LLM candidats sur un espace de compétences. Le routeur exploite alors un signal de préférence agrégé (jugements humains, reward models, signaux contrastifs) pour apprendre, pour chaque requête entrante, une projection vers le modèle attendu comme “le mieux adapté”. Quatre travaux structurent l’état de l’art de cette famille.

2.3.1.0.1 RouteLLM : routage à partir de préférences humaines. L’apport principal de cette famille est le framework ROUTELLM publié par Ong *et al.* à ICLR 2025 [28]. Les auteurs formalisent le routage binaire entre un LLM “fort” et un LLM “faible” comme un problème d’apprentissage à partir de données de préférences issues de Chatbot Arena. Quatre architectures de routeurs y sont comparées : (i) une *Matrix Factorization* sur la matrice (modèle×requête) de scores de victoires ; (ii) un *BERT Classifier* fine-tuné sur les paires d’arena ; (iii) un *Causal LLM Classifier* qui exploite la représentation profonde d’un modèle de fondation ; et (iv) un *Similarity-Weighted ranking* (Similarity-Weighted ranking (SW-ranking)) qui effectue, pour chaque requête entrante q , un calcul d’Elo pondéré par la similarité d’embeddings entre q et les requêtes d’arena pour lesquelles les jugements humains sont disponibles [28]. Plus précisément, en notant E_m le classement Elo du modèle m et $s(q, q_i)$ la similarité cosinus entre l’embedding de q et celui d’une requête d’arena q_i , le score routeur de SW-ranking pour un modèle m s’écrit conceptuellement :

$$R_{\text{SW}}(m, q) = \sum_{i \in \mathcal{A}} s(q, q_i) E_m^{(i)}, \quad (2.1)$$

où $\mathcal{A}$ désigne l’ensemble des requêtes d’arena disponibles et $E_m^{(i)}$ le score Elo de m relativement à cette sous-population de requêtes. SW-ranking constitue ainsi un routeur non-paramétrique qui ne nécessite ni entraînement supervisé dédié ni reward model : seules les préférences d’arena et un modèle d’embedding sont requis [28]. Les auteurs rapportent qu’“avec une calibration sur Chatbot Arena, l’approche réduit le coût d’inférence d’un facteur supérieur à deux sans dégradation de qualité” [28]. Le terme SW-ranking demande une précision, car il prête à confusion : il désigne spécifiquement le routeur Elo-pondéré par similarité d’embeddings de RouteLLM, et non une catégorie générique de routeur.

ZOOTER [29] distille hors-ligne un reward model coûteux en une fonction de routage légère opérant en ligne. Cette logique “oracle coûteux vers routeur léger” préfigure notre couplage Juge $\rightarrow$ estimateur (chapitre 4). ROUTERDC [30] apprend la projection requête $\rightarrow$ modèle par apprentissage contrastif dual, dont l’usage d’embeddings inspire en partie notre estimateur. TRYAGE [31] pousse l’approche à l’extrême avec un LLM-routeur dédié ; son coût d’inférence non négligeable, documenté par les auteurs, motive notre choix d’un classifieur sklearn léger, mesuré à 3,4 ms par intent, plutôt qu’un LLM-routeur.

Ces approches partagent une limite : elles requièrent un corpus de préférences ou un reward model pour calibrer la représentation des modèles. La démarche opposée privilégie la requête.

2.3.2 Approches centrées requête (query-centric)

Les approches query-centric postulent que le prédicteur le plus informatif réside dans la requête : une fois sa difficulté estimée, la sélection se réduit à une correspondance entre difficulté et capacité du modèle. Cette famille fournit l'épine dorsale méthodologique d'InferRouter-LLM.

2.3.2.0.1 FrugalGPT : référence sur le compromis coût-qualité. La contribution la plus citée de cette famille est FRUGALGPT, publiée par Chen *et al.* aux *Transactions on Machine Learning Research* en 2024 [10]. Les auteurs identifient trois leviers de réduction du coût d'inférence : (i) *prompt adaptation* (compression et restructuration du prompt avant invocation) ; (ii) *LLM approximation* (substitution d'un LLM cher par une combinaison cache de réponses + petit modèle de complétion) ; et (iii) *LLM cascade*, principe selon lequel les modèles sont interrogés par ordre croissant de coût, chaque réponse étant validée par un scorer auxiliaire avant escalade éventuelle vers un modèle plus capable.

Les auteurs rapportent que "FrugalGPT peut égaler les performances du meilleur LLM individuel (par exemple GPT-4) avec jusqu'à 98 % de réduction de coût, ou améliorer la précision de GPT-4 de +4 % à coût équivalent" [10]. Sur les benchmarks considérés (HEADLINES, OVERRULING, COQA), le gain se situe entre 50 % et 98 % selon la configuration. FRUGALGPT constitue ainsi la source canonique de toute affirmation chiffrée sur le gain économique d'un routage multi-modèles.

2.3.2.0.2 Hybrid LLM : routage paramétré par un seuil de qualité. Ding *et al.* [32] formalisent à ICLR 2024 une variante *quality-aware* du routage binaire petit/gros modèle. Le routeur, fondé sur un encodeur BERT, prédit pour chaque requête une *difficulty score* définie comme la probabilité que le petit modèle soit aussi bon que le gros sur cette requête ; une politique à seuil ajustable au test-time exploite ce score pour arbitrer. Les auteurs distinguent trois variantes du routeur (déterministe, probabiliste, probabiliste à transformation) et montrent qu'on peut économiser jusqu'à 40 % des invocations du gros modèle sans dégradation de qualité sur le benchmark MixInstruct [32]. Cette formalisation explicite d'une notion de difficulté intrinsèque à la requête, séparée de la notion de coût ou de qualité du modèle, est directement réutilisée dans la définition de l'estimateur de complexité d'InferRouter-LLM (chapitre 3).

AUTOMIX [33] prolonge la cascade par une auto-évaluation few-shot arbitrée par un *meta-verifier* Partially Observable Markov Decision Process (POMDP), avec une réduction de coût supérieure à 50 % à performance comparable. Sa démonstration de la viabilité d'une auto-évaluation intégrée inspire notre module Juge, sous une forme distincte (juge externe indépendant). LLM-BLENDER [34] offre un contrepoint : en invoquant tous les modèles puis en les fusionnant (*PairRanker* et *GenFuser*), il maximise la qualité mais sacrifie l'économie de coût, incompatible avec nos contraintes de latence. EAGLE [35] montre enfin qu'un routeur efficace à base d'Elo peut être construit sans entraînement supervisé ; nous mobilisons ce constat en §2.5 pour justifier que notre recours à l'entraînement tient à la spécificité du domaine réseau, non à une nécessité générique.

2.3.2.0.3 Routage à partir de benchmarks publics. Shnitzer *et al.* [36] reformulent à COLM 2024 le problème de sélection d'un LLM comme un problème d'apprentissage entraînable à partir des benchmarks publics existants (MMLU, HellaSwag, etc.). La sélection d'un modèle optimal pour une tâche inconnue est réduite à un ensemble de tâches de classification binaire ("le modèle m va-t-il réussir cette requête ?") apprises sur les scores des benchmarks. Les auteurs montrent une amélioration consistante par rapport à la sélection uniforme du *single-best model*, tout en pointant le risque

2.4. ÉVALUATION AUTOMATIQUE DE QUALITÉ (LLM-AS-A-JUDGE)

d’overfitting aux benchmarks [36]. Cette démonstration, à savoir qu’un router performant peut être appris à partir d’un volume modéré de données étiquetées, est essentielle pour InferRouter-LLM, qui s’appuie sur un dataset de quelques centaines d’intents réseau générés par LLM, annotés en domaine, complexité et criticité (chapitre 5).

L’ensemble des approches query-centric converge vers un schéma commun : l’estimation préalable d’une difficulté par requête autorise un routage efficace, et le compromis coût-qualité peut être rendu pilotable par un seuil ajustable. Ce schéma constitue le socle méthodologique d’InferRouter-LLM ; il est cependant insuffisant pour caractériser de manière structurée l’ensemble de la littérature, qui mélange souvent les deux logiques. La sous-section suivante propose une taxonomie à trois axes destinée à y remédier.

2.3.3 Taxonomie révisée

Pour positionner ces travaux de manière discriminante, nous retenons trois axes, en prolongement de la formalisation unifiée de Dekoninck *et al.* [37] qui ramène routing et cascading à un même problème d’optimisation sous contrainte de coût. La *granularité* oppose une décision par requête (*per-query*), adoptée par la quasi-totalité des travaux, à une décision par session (*per-session*), réservée aux systèmes agentiques comme ECOASSISTANT [38]. La *source d’information* distingue les approches model-centric (cartographie apprise des modèles [28, 29, 30, 31]), query-centric (difficulté intrinsèque de la requête [32, 10, 36, 39]) et hybrides à retour d’exécution [33, 37]. L’*adaptation dans le temps* sépare enfin les routeurs statiques des routeurs adaptatifs, par cascade hybride [10] ou apprentissage en ligne [38, 33]. InferRouter-LLM se classe en *per-query*, query-centric ancré au domaine, et adaptatif via l’auto-calibration par juge local (chapitre 4).

Le tableau 2.1 (§2.5) positionne les travaux discutés sur ces trois axes et sur les dimensions du cas réseau. Il fait ressortir la combinaison rare que vise InferRouter-LLM : granularité *per-query*, source query-centric ancrée dans un domaine spécialisé, et adaptation en ligne par auto-calibration via un juge local.

2.4 Évaluation automatique de qualité (LLM-as-a-Judge)

Tout routeur multi-LLM s’appuie sur une estimation de la qualité attendue de chaque candidat, notée $\hat{q}_i(e)$ au chapitre 3 (éq. (3.4)) et calibrée par la qualité $q_i(e)$ mesurée a posteriori. L’hypothèse H2 (§ 3.3.5) postulant l’absence d’opérateur humain dans la boucle, il faut une procédure d’évaluation de $q_i(e)$ à la fois fiable pour piloter le routage et assez légère pour ne pas annuler le gain économique. Deux familles s’y prêtent : les métriques lexicales classiques (BLEU, ROUGE, BERTScore) et le paradigme LLM-as-a-Judge, dont le survey de Gu *et al.* [40] distingue les formats (pointwise, pairwise, listwise) et les biais.

2.4.1 Méthodes classiques (BLEU, ROUGE, BERTScore)

Trois métriques structurent historiquement l’évaluation automatique de texte. Bilingual Evaluation Understudy (BLEU) (Papineni *et al.*, 2002) mesure la précision des n-grammes communs à une référence ; Recall-Oriented Understudy for Gisting Evaluation (ROUGE) (Lin, 2004) en renverse la perspective vers le rappel ; BERTSCORE [41] remplace la comparaison lexicale par une similarité

2.4. ÉVALUATION AUTOMATIQUE DE QUALITÉ (LLM-AS-A-JUDGE)

d’embeddings contextuels et améliore la corrélation avec l’humain. Le détail de ces métriques figure en annexe A.4.

Ces trois métriques restent inadaptées à l’évaluation d’une réponse sur un intent réseau, pour trois raisons. Le biais lexical pénalise les paraphrases sémantiquement équivalentes, fréquentes quand plusieurs configurations distinctes réalisent le même comportement. L’exigence d’une réponse de référence unique ne tient pas pour des intents admettant plusieurs réalisations valides. Enfin, la qualité d’une configuration engage des dimensions (correction syntaxique, conformité à la politique, contraintes du domaine) qu’aucun score de proximité textuelle n’isole. Cette inadéquation [21, 22] motive le recours au paradigme LLM-as-a-Judge.

2.4.2 LLM-as-a-Judge (G-Eval, RocketEval ICLR 2025)

Le paradigme LLM-as-a-Judge désigne l’utilisation d’un LLM lui-même comme évaluateur automatique d’une sortie textuelle produite par un autre LLM (ou par le même). Il émerge à partir de 2023 comme une réponse directe aux limites des métriques classiques : le juge LLM, par sa capacité de raisonnement généraliste, peut intégrer plusieurs dimensions de qualité sans nécessiter de référence figée et accepter les variations paraphrastiques légitimes [40]. Cinq contributions structurent cet état de l’art, dont la dernière, ROCKETEVAL, est retenue comme méthode de jugement d’InferRouter-LLM.

Quatre travaux tracent la trajectoire du paradigme. G-EVAL [42] formalise l’usage de GPT-4 comme juge par génération d’une chaîne de raisonnement puis notation guidée, atteignant une corrélation de Spearman de 0,514 avec l’humain. Zheng *et al.* [43] valident empiriquement le paradigme (agrément GPT-4/humain supérieur à 80 %, comparable à l’accord inter-annotateurs) et documentent quatre biais récurrents à mitiger : position, verbosité, auto-favoritisme et raisonnement numérique. JUDGE LLM [44] et PANDALM [45] montrent que la fonction de jugement se transfère vers des modèles open-source légers (agrément supérieur à 90 % avec le teacher pour JudgeLM), condition d’un déploiement local sans dépendance API. Ces approches partagent deux limites : un format *pointwise* ou *pairwise* peu explicable, et un coût d’inférence qui reste significatif à l’échelle d’une boucle d’auto-calibration. ROCKETEVAL répond à cette double limitation.

2.4.2.0.1 RocketEval : évaluation par checklist instanciée. Wei *et al.* [12] présentent à ICLR 2025 le framework ROCKETEVAL, méthode adoptée par InferRouter-LLM pour l’évaluation automatique de qualité. Le principe en est le suivant : transformer l’évaluation forme-libre d’une réponse en une série de questions binaires Y/N (une *checklist*) générées en amont à partir du prompt initial. L’évaluation s’organise en trois étapes : (i) *Checklist Creation*, où un LLM puissant génère, pour chaque prompt, une liste de critères vérifiables spécifiques à l’instance ; (ii) *Checklist Grading*, où un LLM léger note indépendamment chaque item Y/N sur la réponse candidate, sans avoir à comparer plusieurs candidats simultanément ; (iii) *Score Prediction*, qui agrège les jugements binaires en un score final via une régression supervisée [12].

L’architecture sépare deux coûts que les approches directes confondent. La génération de la checklist demande un raisonnement coûteux, mais elle n’a lieu qu’une fois par prompt ; la notation en ligne se réduit à des questions binaires, tâche simple qu’un modèle léger suffit à traiter. Cette dissociation rend la méthode linéairement scalable et quasi insensible au biais de position, la checklist fournissant de surcroît une trace explicable du jugement.

Le résultat empirique central est particulièrement adapté à notre contexte : “ROCKETEVAL, utilisant Gemma-2-2B comme juge, atteint une corrélation de 0,965 avec les préférences humaines, comparable

à celle obtenue par GPT-4o, tout en réduisant le coût d'évaluation à grande échelle d'un facteur supérieur à 50" [12]. Cette quasi-équivalence avec GPT-4o sur un modèle de 2 milliards de paramètres exécutable en local change radicalement l'économie d'une évaluation en boucle fermée : la qualité $q_i(e)$ devient mesurable sans coût marginal d'API, ce qui rend tractable l'invocation systématique du juge à chaque itération d'auto-calibration du Complexity Estimator (cf. chapitre 4). C'est cette propriété, une corrélation élevée à GPT-4 pour un modèle exécutable localement, qui justifie le choix de ROCKETEVAL comme socle de la couche d'évaluation d'InferRouter-LLM, instanciée dans notre prototype par `gemma2` servi par Ollama (cf. chapitre 4). Le chapitre 5 confronte ce socle au domaine réseau et en délimite la portée réelle.

En conjuguant des routers adaptatifs (§ 2.3) et des juges automatiques fiables (§ 2.4), une nouvelle génération d'architectures peut envisager un routage dynamique multi-LLM dans des contextes réseau exigeants. La section suivante synthétise ce paysage et positionne InferRouter-LLM par rapport au *gap* méthodologique identifié.

2.5 Synthèse et positionnement

Les quatre sections précédentes convergent. L'IBN est mature mais entravé par la carence de traduction lexicale et d'évaluation automatique (§2.1). Les LLM appliqués aux réseaux lèvent la première limitation mais mobilisent un modèle cloud unique sans arbitrage (§2.2). Le routage dynamique fournit le cadre de cet arbitrage (§2.3), et ROCKETEVAL [12] débloquent l'évaluation automatique à coût compatible avec une boucle fermée (§2.4). Nous en tirons le *gap* méthodologique et le positionnement d'InferRouter-LLM.

2.5.1 Comparatif des routers existants

Le tableau 2.1 confronte les travaux représentatifs à InferRouter-LLM selon six dimensions : granularité, source d'information, adaptation temporelle, domaine, évaluation automatique de qualité et contraintes coût-latence. Il rend visibles quatre régularités structurelles.

D'abord, la couverture par domaine est presque uniformément générique : la quasi-totalité des routers sont validés sur des benchmarks de questions ouvertes (Chatbot Arena, MMLU, MixInstruct) sans ancrage réseau [28, 10, 32, 34, 33, 30]. Seul SEMANTIC ROUTER [20] vise un domaine réseau, mais pour sélectionner un composant (fonction du cœur 5G), non un LLM cible. Ensuite, aucun dispositif ne mesure la qualité réelle de la réponse a posteriori : tous s'appuient sur des proxys (préférences hors-ligne, score appris, gating de confiance) corrélés à la qualité mais ne la mesurant pas à l'instance. Enfin, aucun n'intègre simultanément des bornes dures de coût et de latence par requête ; au plus un seuil réglable sur un seul critère [10, 32, 28]. Ces trois absences, formalisées par les contraintes $c_i \leq C_{\max}(e)$ et $\ell_i(e) \leq L_{\max}(e)$ (éq. (3.5)), définissent l'espace laissé vacant : InferRouter-LLM couple une source query-centric ancrée au domaine réseau, un LLM-Juge local d'évaluation post-hoc [12] et un arbitrage tri-critère sous bornes dures.

2.5.2 Gap identifié et justification d'InferRouter-LLM

Le triple verrou dégage ci-dessus (absence de spécialisation réseau, d'évaluation factuelle a posteriori et d'arbitrage sous bornes dures) n'est pas un raffinement académique : il répond à trois exigences

2.5. SYNTHÈSE ET POSITIONNEMENT

Table 2.1: Comparaison des routers LLM existants face à InferRouter-LLM, sur six dimensions discriminantes. La dernière colonne indique la prise en compte de bornes dures de coût et de latence par requête.

Router	Granul.	Source d'info.	Adapt.	Domaine	Eval. auto. qualite	Cout-lat.
RouteLLM [28]	per-query	Model-centric	statique	Generique	Win-rate Arena (offline)	✓
Zooter [29]	per-query	Model-centric	statique	Generique	Reward model distille	–
RouterDC [30]	per-query	Model-centric	statique	Generique	Implicite (contrastif)	–
Tryage [31]	per-query	Model-centric	statique	Generique	Prediction de perf.	✓
Eagle [35]	per-query	Model-centric	statique	Generique	Elo global + local	–
FrugalGPT [10]	per-query	Query + execution	cascade	Generique	Scorer de validation	✓
HybridLLM [32]	per-query	Query-centric	statique	Generique	Difficulty probabiliste	✓
AutoMix [33]	per-query	Query + execution	cascade	Generique	Meta-verifier POMDP	✓
LLM-Blender [34]	per-query	Execution (ensembl.)	statique	Generique	PairRanker (pairwise)	–
Shnitzer <i>et al.</i> [36]	per-query	Query-centric	statique	Generique	Scores de benchmarks	–
EcoAssistant [38]	per-session	Execution	en-ligne	Generique	Escalade de cascade	✓
IRT-Router [39]	per-query	Query-centric	statique	Generique	Difficulte latente (IRT)	–
Manias Semantic [20]	per-query	Query-centric	statique	Specialise (5G)	Pre-filtrage semantique	–
InferRouter-LLM	per-query	Query-centric (dom.)	en-ligne	Specialise (reseau)	LLM-Judge local	✓

opérationnelles documentées. La cadence des intents 5G, plusieurs centaines par jour en orchestration de slices [15, 8], exclut toute boucle humaine d'évaluation et impose un juge automatique fiable et économique [12]. Le coût des LLM cloud à l'échelle des opérations, ainsi que l'inobservabilité de leur charge (hypothèse H3, §3.3.5), rendent une architecture mixte local/cloud plus défendable qu'une invocation cloud uniforme [18]. Enfin, les SLA réseau imposent des bornes dures par intent (latence uRLLC bornée par les spécifications 3GPP [8], budget borné par l'opérateur), incompatibles avec une optimisation en moyenne [28, 10].

InferRouter-LLM répond à ce verrou par trois contributions, détaillées au chapitre 4 : un estimateur de complexité sémantique d'intents réseau répondant en 3,4 ms, un routeur sélectionnant sur l'ensemble admissible $M(e)$ le modèle le moins coûteux dont la qualité attendue $\hat{q}_i(e)$ atteint un plancher indexé sur la criticité (équ. (3.5) et (3.6)), et un LLM-Juge local appliquant ROCKETEVAL [12] pour l'auto-calibration. Chacune adresse une facette du verrou ; l'évaluation (chapitre 5) en délimite la portée réelle, notamment celle du juge, fiable en discrimination grossière mais non en discrimination fine.

Le chapitre suivant formalise rigoureusement ce problème de routage d'intents réseau : il pose les ensembles, fonctions et contraintes du modèle, énonce les hypothèses de travail (notamment H3 sur l'inobservabilité de la charge cloud), et fixe les notations qui structurent les trois contributions ci-dessus.

Chapter 3

Définition du problème

La Table 3.1 regroupe les notations du cadre formel utilisées dans ce chapitre et les suivants. Les sections 3.1 et 3.2 mobilisent certains symboles avant leur définition formelle (§3.3) ; le tableau donne le renvoi correspondant, sans se substituer aux définitions en prose.

Table 3.1: Notations du cadre formel d’InferRouter-LLM.

Symbole	Signification	Définition
e	intent réseau exprimé en langage naturel	éq. (3.1)
$d(e)$	domaine fonctionnel de l’intent	éq. (3.1)
$\mathcal{D}$	ensemble des domaines fonctionnels	éq. (3.2)
$\kappa(e)$	criticité opérationnelle (low/med/high)	éq. (3.1)
$L_{\max}(e)$	latence maximale tolérée pour e (ms)	éq. (3.1)
$C_{\max}(e)$	coût maximal admissible par inférence	éq. (3.1)
$n(e)$	nombre d’entités réseau distinctes de l’énoncé	§3.2.1
$p(e)$	profondeur d’inférence requise	§3.2.2
$\delta(e)$	sous-ensemble des domaines touchés par e	§3.2.3
M	pool de LLM cibles $\{m_1, \dots, m_k\}$	éq. (3.3)
$M(e)$	sous-ensemble des modèles admissibles pour e	éq. (3.5)
$R^*(e)$	modèle sélectionné par le routeur	éq. (3.6)
$\hat{q}_i(e)$	estimation a priori de la qualité de m_i sur e	éq. (3.4)
$q_i(e)$	qualité mesurée a posteriori par le LLM Juge	§3.3.2
$q_{\min}(\kappa(e))$	plancher de qualité exigé, croissant en criticité	éq. (3.6)
c_i	coût unitaire d’inférence de m_i (proxy tarifaire)	éq. (3.4)
$\ell_i(e)$	latence d’inférence de m_i sur e (ms)	éq. (3.4)
$T(e)$	temps total de traitement de e ($T_{\text{est}} + T_{\text{rout}} + T_{\text{inf}} + T_{\text{juge}}$)	éq. (3.8)
AIQ	qualité moyenne d’inférence sur le jeu d’évaluation	éq. (3.7)

3.1 Modélisation (architecture) système

Avant toute formalisation, nous décrivons l'architecture cible du système et le rôle de chacun de ses composants. InferRouter-LLM reçoit des intents réseau en langage naturel et les achemine vers un modèle de langage adapté, sous contrainte de qualité, de latence et de coût. Le système s'articule autour d'un routeur central, d'un pool de LLM cibles, d'un LLM Juge local et d'un estimateur de complexité. L'ensemble est pensé pour un déploiement edge, au plus près du contrôleur réseau.

3.1.1 Vue d'ensemble des composants

Le routeur est l'organe de décision : il reçoit un intent textuel et choisit le LLM cible, sans exécuter lui-même d'inférence. Sa décision croise trois signaux : les caractéristiques sémantiques fournies par l'estimateur de complexité, les attributs de coût et de latence exposés par le pool, et la qualité mesurée a posteriori par le Juge, qui alimente la calibration. Cette séparation autorise l'ajout ou le retrait d'un candidat sans toucher au cœur du routage. Le pool mêle des LLM génériques, repli sur les intents multi-domaines, et des LLM spécialisés par domaine (RAN, cœur, sécurité, slice).

3.1.1.0.1 LLM spécialisé. Le terme désigne un modèle du pool affecté à un domaine, attendu plus précis à coût souvent moindre. Il n'implique aucun comportement agentique : pas d'appel d'outils ni d'action autonome. La Figure 3.1 résume cette organisation.

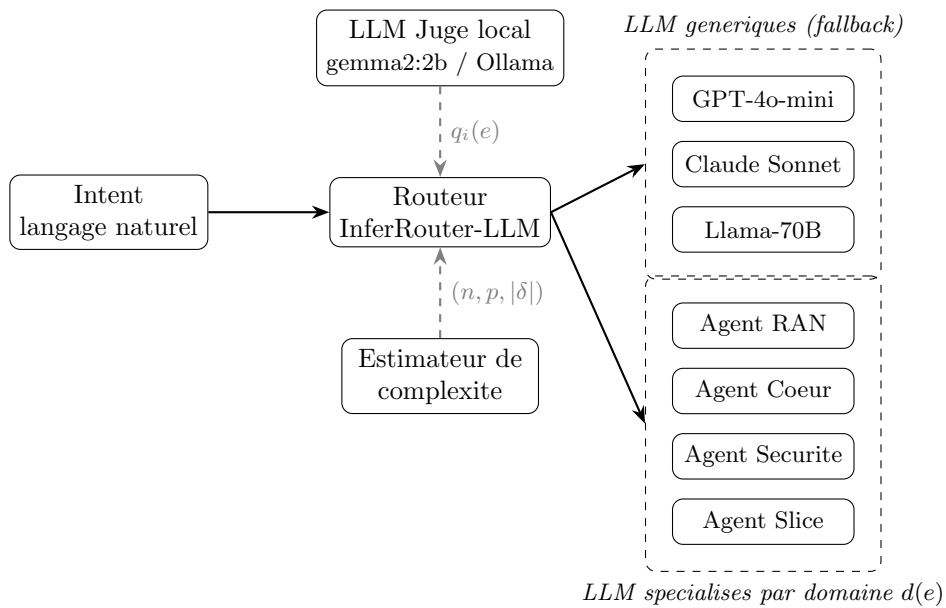


Figure 3.1: Architecture du pool de LLM cibles d'InferRouter-LLM. Le routeur sélectionne entre des LLM génériques (fallback multi-domaine) et des LLM spécialisés par domaine fonctionnel (RAN, cœur, sécurité, slice). Le LLM Juge local fournit la qualité estimée $q_i(e)$ a posteriori ; l'estimateur de complexité fournit les attributs $(n(e), p(e), |\delta(e)|)$.

Le Juge évalue la qualité par la méthode RocketEval [12], exécutée localement (Ollama, **gemma2**), pour deux raisons : une latence d'évaluation maîtrisée et le maintien des intents sensibles dans le périmètre

3.1. MODÉLISATION (ARCHITECTURE) SYSTÈME

de l'opérateur. Son rôle est nuancé par le chapitre 5 : il départage de manière fiable des candidats nettement différents (discrimination grossière), ce que le routage demande, mais pas une erreur isolée dans une réponse sinon correcte (§5.3).

3.1.1.0.2 Déploiement edge. Le routeur et le Juge sont co-localisés avec le contrôleur IBN en bordure, ce qui rend le transport intra-cluster négligeable devant l'inférence des LLM cibles (§3.3.7). Ces cibles sont appelées en cloud via OpenRouter : leur charge Graphics Processing Unit (GPU) n'est pas observable, et la décision s'appuie sur des grandeurs côté client (latence mesurée, tarif, qualité estimée). La Figure 3.2 détaille les étapes et les composantes de latence. Le détail d'implémentation de ces composants figure au chapitre 4.

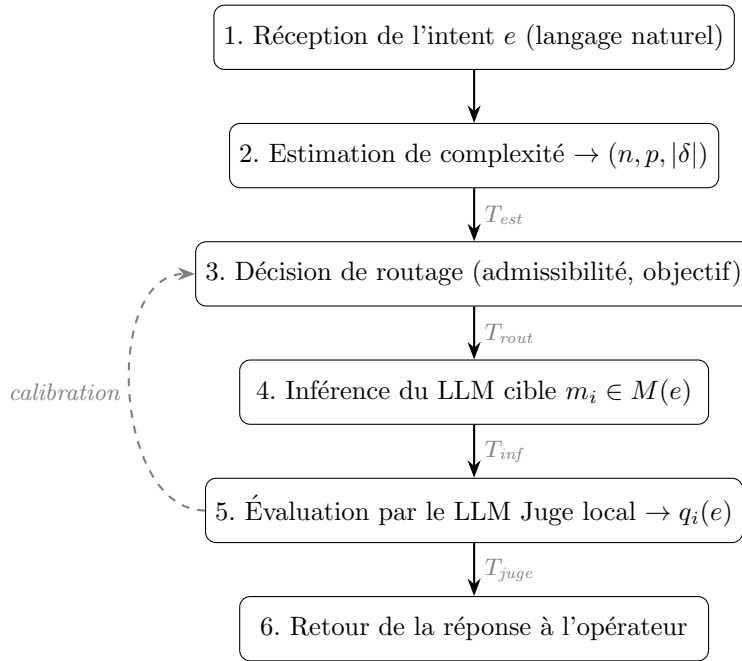


Figure 3.2: Flux de traitement d'un intent par InferRouter-LLM. Les annotations T_{est} , T_{rout} , T_{inf} et T_{juge} matérialisent les composantes de la décomposition de latence (Equation 3.8). La latence de transport réseau est absorbée par la co-localisation du routeur en bordure de l'infrastructure supervisée.

3.1.2 Comportement attendu par slice réseau

Chaque type de slice impose son régime de qualité de service. La spécification 3GPP TS 22.261 [46] définit trois familles, dont découlent des répercussions distinctes sur la criticité $\kappa(e)$ et le budget de latence $L_{\max}(e)$ de l'intent (éq. (3.1)). L'*enhanced Mobile BroadBand* (eMBB) vise le haut débit à latence tolérante et criticité moyenne, laissant l'arbitrage coût-qualité libre. L'*Ultra-Reliable Low-Latency Communications* (URLLC) exige une latence très basse et une haute criticité : le budget $L_{\max}(e)$ se resserre et restreint l'ensemble admissible $M(e)$ aux modèles les plus rapides. Le *massive Machine-Type Communications* (mMTC) connecte un grand volume de terminaux à faible débit : la

3.2. CHALLENGES DU SYSTÈME

criticité unitaire est modérée, mais le coût agrégé domine et privilégie un LLM léger. La Table 3.2 résume cette dépendance.

Une précision s'impose sur la nature de $L_{\max}(e)$, sous peine de confusion avec les ordres de grandeur cités ci-dessus. Le routeur opère dans le plan de contrôle : il traite une demande d'administration formulée par un opérateur, dont le budget se compte en secondes, voire en minutes pour un intent de planification. La milliseconde exigée par l'uRLLC concerne le plan de données, c'est-à-dire le trafic que la slice transporte une fois configurée. Un intent URLLC resserre donc $L_{\max}(e)$ parce qu'une demande touchant un service critique doit être servie promptement, et non parce que le LLM devrait répondre en une milliseconde, ce qu'aucun modèle génératif ne permet.

La criticité $\kappa(e)$ agit sur le routage par deux voies. Elle conditionne les bornes $L_{\max}(e)$ et $C_{\max}(e)$ qui définissent l'ensemble admissible [8], et elle fixe le plancher de qualité $q_{\min}(\kappa(e))$ qui apparaît explicitement dans l'objectif (éq. (3.6)) : c'est par elle que l'opérateur décide où il accepte de payer davantage.

Table 3.2: Répercussion du type de slice sur la criticité $\kappa(e)$ et les bornes de l'intent (régimes qualitatifs dérivés des exigences de service de la spécification 3GPP TS 22.261 [46]).

Famille de slice	$\kappa(e)$	Régime de $L_{\max}(e)$	Borne dominante
eMBB	med	tolérant (ordre de la seconde)	aucune (arbitrage libre)
URLLC	high	resserré	$L_{\max}(e)$
mMTC	med	tolérant	$C_{\max}(e)$

3.2 Challenges du système

Le routage d'intents réseau soulève des verrous absents du routage généraliste : la complexité d'un intent est difficile à mesurer a priori, aucun oracle humain n'est disponible au runtime, et les contraintes varient d'une slice à l'autre. Cette section traite le premier verrou, caractériser la complexité par des grandeurs observables. L'estimateur (chapitre 4) doit opérationnaliser une difficulté propre aux intents réseau, distincte de la difficulté "académique" des routeurs généralistes [39, 32]. Trois critères observables servent d'entrées au classifieur.

3.2.1 Nombre d'entités impliquées

Soit $n(e)$ le nombre d'entités réseau distinctes mentionnées dans un intent e , une entité désignant une slice, une fonction réseau (NF), un Key Performance Indicator (KPI) ou une politique de sécurité nommée. L'intent "quelle est la latence de la slice URLLC #1 ?" implique $n(e) = 2$; un intent de reconfiguration multi-slices sous SLA et politique de sécurité atteint $n(e) \geq 5$. Ce nombre mesure la surface de raisonnement à couvrir, suivant la logique du "difficulty score" de Ding *et al.* [32], et se quantifie par un module NER spécialisé réseau.

3.2.2 Profondeur d'inférence requise

Soit $p(e) \in \mathbb{N}^*$ la profondeur d'inférence, définie comme le nombre d'étapes de raisonnement nécessaires à la réponse. Une lecture d'état directe correspond à $p(e) = 1$; un intent d'optimisation sous contraintes

3.3. FORMULATION DU PROBLÈME

(diagnostic, calcul d’un plan, vérification de non-régression) atteint $p(e) \geq 3$. Cette difficulté intrinsèque fait écho à la variable latente du modèle Item Response Theory (IRT) de Song *et al.* [39].

La profondeur n’est pas observable directement : nous l’approchons par le nombre de contraintes de l’énoncé, où une contrainte est toute condition qui restreint la réponse. Deux formes sont comptées : les marqueurs d’objectif (“tout en respectant”, “sans dégrader”) et les seuils chiffrés (“latence inférieure à 10 ms”). Ce comptage par expressions régulières définit l’attribut `n_constraints` exploité par l’estimateur (chapitres 4 et 5).

3.2.3 Domaines croisés

Soit $\delta(e) \subseteq \mathcal{D}$ le sous-ensemble des domaines fonctionnels concernés par l’intent, projeté sur les quatre catégories de l’équation (3.2). Un intent monodomaine ($|\delta(e)| = 1$) s’adresse au LLM spécialisé du domaine $d(e)$; un intent multi-domaine ($|\delta(e)| \geq 2$, par exemple `RAN` $\cap$ sécurité) requiert un généraliste ou une orchestration entre spécialisés (chapitre 6). Cette dimension, spécifique aux intents réseau, n’est pas abordée par la littérature de routage [37].

3.2.4 Opérationnalisation

Les trois critères ($n(e), p(e), |\delta(e)|$) sont les variables explicatives observables de la complexité. Leur agrégation est confiée à un classifieur léger (`sentence-transformers` puis `scikit-learn`, chapitre 4), sous une cible de conception de moins de 15 ms par estimation, négligeable devant le budget $L_{\max}(e)$ d’un intent (de l’ordre de la seconde) et vérifiée au chapitre 5.

3.3 Formulation du problème

Le cadre mathématique du routage s’inspire des formulations canoniques qualité/coût [32, 37] et de la projection de RouterBench [47], en y ajoutant les contraintes propres au contexte réseau : criticité opérationnelle, latence dure, domaine fonctionnel.

3.3.1 Modélisation d’un intent réseau

Soit E l’ensemble des intents réseau exprimés en langage naturel par un opérateur. Un intent $e \in E$ désigne une requête de configuration, d’optimisation ou de diagnostic portant sur une infrastructure réseau (par exemple, “créer une slice URLLC à faible latence pour véhicules connectés”). Chaque intent est caractérisé par le quadruplet d’attributs :

$$e \equiv (d(e), \kappa(e), L_{\max}(e), C_{\max}(e)), \quad (3.1)$$

où :

- $d(e) \in \mathcal{D}$ désigne le domaine fonctionnel de l’intent, avec

$$\mathcal{D} = \{\text{RAN}, \text{cœur}, \text{sécurité}, \text{slice}\}; \quad (3.2)$$

- $\kappa(e) \in \{\text{low}, \text{med}, \text{high}\}$ représente le niveau de criticité opérationnelle de l’intent, reflétant l’impact potentiel d’une réponse erronée sur le service réseau ; son effet sur les bornes $L_{\max}(e)$ et $C_{\max}(e)$ est résumé à la Table 3.2 ;

3.3. FORMULATION DU PROBLÈME

- $L_{\max}(e) \in \mathbb{R}_{>0}$ dénote la latence maximale tolérée pour traiter l'intent e , exprimée en millisecondes ;
- $C_{\max}(e) \in \mathbb{R}_{>0}$ correspond au coût maximal admissible par inférence, exprimé comme un proxy adimensionnel dont la définition opérationnelle est donnée en §3.3.6.

3.3.2 Pool de modèles cibles

Le système dispose d'un pool fini de LLM spécialisés par domaine, noté

$$M = \{ m_1, m_2, \dots, m_k \}, \quad k \in \mathbb{N}^*, \quad (3.3)$$

dont la composition concrète sera précisée au chapitre 4. À chaque modèle $m_i \in M$ et chaque intent $e \in E$, on associe le triplet caractéristique :

$$m_i \longleftrightarrow (\hat{q}_i(e), c_i, \ell_i(e)), \quad (3.4)$$

où :

- $\hat{q}_i(e) \in [0, 1]$ désigne l'estimation a priori de la qualité de la réponse du modèle m_i pour l'intent e , disponible avant toute inférence. Elle est initialisée puis mise à jour par la calibration hors ligne décrite au chapitre 4 ;
- $c_i \in \mathbb{R}_{>0}$ représente le coût unitaire d'inférence du modèle m_i , supposé indépendant de l'intent (proxy basé sur le tarif par jeton, cf. §3.3.6) ;
- $\ell_i(e) \in \mathbb{R}_{>0}$ dénote la latence d'inférence observée pour le modèle m_i sur l'intent e , en millisecondes.

La qualité effective de la réponse, notée $q_i(e) \in [0, 1]$, est mesurée a posteriori par le LLM Juge local (méthode RocketEval, checklist générée par intent puis gradée, chapitre 4). Le routeur ne l'observe jamais au moment de la décision : elle sert de référence pour calibrer $\hat{q}_i(e)$ et pour évaluer le système au chapitre 5, où elle est exploitée en comparaison relative entre candidats. Cette distinction entre les deux notations lève la circularité apparente entre décision et évaluation : la décision repose sur $\hat{q}_i(e)$, la mesure $q_i(e)$ n'intervient qu'après inférence.

Contrairement à la formulation de Dekoninck et al. [37] qui intègre uniquement une contrainte de coût, nous explicitons une contrainte de latence par intent, nécessaire dans un contexte opérationnel réseau où les SLA imposent des bornes dures.

3.3.3 Critères d'admissibilité

Pour un intent e donné, on définit le sous-ensemble admissible des modèles satisfaisant simultanément les contraintes de latence et de coût :

$$M(e) = \{ m_i \in M : \ell_i(e) \leq L_{\max}(e) \wedge c_i \leq C_{\max}(e) \}. \quad (3.5)$$

L'équation (3.5) définit une enveloppe faisable similaire à celle décrite par RouterBench [47] dans le plan coût-qualité, en y ajoutant la dimension temporelle. Le filtrage par domaine fonctionnel $d(e)$

3.3. FORMULATION DU PROBLÈME

peut, selon l'implémentation, intervenir en amont (restriction du pool M aux LLM spécialisés du domaine $d(e)$) ou en aval via une pénalité dans $\hat{q}_i(e)$; ce point est traité en §3.2.

3.3.4 Objectif du routeur

Le routeur ne maximise pas la qualité. Face à un modèle lourd nettement supérieur au léger, l'argmax de qualité sélectionne toujours le lourd, et le routage dégénère en stratégie ALWAYS-HEAVY sans aucun arbitrage. Nous adoptons la formulation duale, qui correspond à un SLA opérationnel : garantir une qualité minimale acceptable, puis servir l'intent au moindre coût.

Soit $q_{\min}(\kappa(e))$ le plancher de qualité exigé pour l'intent e , fonction croissante de sa criticité $\kappa(e)$: un intent critique impose un plancher plus élevé, donc bascule plus volontiers vers le lourd. Le routeur, application $R : E \rightarrow M \cup \{\perp\}$, retient parmi les modèles admissibles au sens des SLA (éq. (3.5)) qui atteignent ce plancher le modèle de coût minimal :

$$R^*(e) = \arg \min_{\substack{m_i \in M(e) \\ \hat{q}_i(e) \geq q_{\min}(\kappa(e))}} c_i. \quad (3.6)$$

C'est le renversement primal-dual de la maximisation de qualité sous budget : au lieu de “la meilleure qualité sous contrainte de coût”, “le moindre coût sous contrainte de qualité”. Le plancher $q_{\min}$ est le paramètre de contrôle de l'opérateur : le faire varier décrit la frontière de Pareto coût-qualité du système, du tout-léger (plancher bas) au tout-lourd (plancher haut), évaluée au chapitre 5 (§5.5). La sélection repose sur l'estimation a priori $\hat{q}_i(e)$, calibrée hors ligne par les mesures a posteriori $q_i(e)$ du Juge (§3.3.2).

3.3.4.0.1 Plancher inatteignable. Si aucun modèle admissible n'atteint $q_{\min}(\kappa(e))$, le routeur ne refuse pas l'intent : il retient au mieux le modèle admissible de plus haute qualité. Le plancher est une cible, non une contrainte dure ; seuls les SLA de latence et de coût le sont.

3.3.4.0.2 Cas dégénéré. Lorsque le sous-ensemble admissible $M(e)$ défini par l'équation (3.5) est vide au sens des SLA, le routeur retourne le statut “intent non routable”, noté $\perp$, plutôt que de violer un SLA. Le système renvoie une erreur explicite à l'opérateur, contenant le motif de non-admissibilité : dépassement de $L_{\max}(e)$, de $C_{\max}(e)$, ou des deux. Ce comportement diffère des cadres généralistes [32, 37] qui supposent implicitement $M(e) \neq \emptyset$. Dans un contexte réseau, la non-réponse est préférable à une réponse qui viole un SLA.

3.3.4.0.3 Résolution des égalités. Si plusieurs modèles atteignent le coût minimal dans l'équation (3.6), le routeur départage par la latence $\ell_i(e)$ la plus faible. Ce critère de départage est conforme à la projection Pareto-optimale du plan coût-qualité décrite par Hu et al. [47].

3.3.5 Hypothèses opérationnelles

La formulation précédente repose sur quatre hypothèses opérationnelles, que nous explicitons pour circonscrire le périmètre de validité de nos contributions et préciser les variables contrôlées au moment de l'évaluation.

3.3. FORMULATION DU PROBLÈME

3.3.5.0.1 H1 – Spécialisation par domaine. Pour chaque domaine fonctionnel $d \in \mathcal{D}$ (éq. (3.2)), nous supposons qu’un unique LLM spécialisé couvre le domaine, à l’exclusion de toute stratégie d’ensembling ou de cascade intra-domaine. Cette hypothèse isole la variable spécialisation de la variable architecture du modèle dans l’analyse expérimentale du chapitre 5. Elle est conforme aux protocoles comparés de la littérature de routage qualité/coût, où Ding et al. [32] et Dekoninck et al. [37] évaluent chacun un seul candidat par classe de complexité. La généralisation à un ensemble multi-modèle par domaine est laissée à des travaux ultérieurs.

3.3.5.0.2 H2 – Absence d’oracle humain au runtime. La mesure de qualité $q_i(e)$ qui calibre l’estimation $\hat{q}_i(e)$ (§3.3.2) est produite exclusivement par le LLM Juge local, sans intervention humaine au moment de l’inférence. Cette hypothèse répond à la contrainte temps réel des intents réseau : un opérateur ne peut tolérer un délai de validation humaine pour une reconfiguration de slice ou une réponse à un incident de sécurité. Elle suit la même logique que les évaluateurs automatisés adoptés par RouterBench [47], en se distinguant par une exécution locale du Juge afin de maîtriser coût et confidentialité.

3.3.5.0.3 H3 – Charge système non observable en cloud. Les LLM cibles du pool M (éq. (3.3)) sont majoritairement accessibles via des API cloud propriétaires (typiquement OpenRouter), ce qui rend la charge GPU/CPU sous-jacente non observable depuis le routeur. Aucune décision de routage ne peut donc s’appuyer sur des indicateurs de saturation infrastructurelle, comme ce serait le cas dans une formulation de placement de charge classique. Cette hypothèse justifie le choix de fonder la décision sur des grandeurs observables côté client (latence mesurée $\ell_i(e)$, tarif unitaire c_i , estimation de qualité $\hat{q}_i(e)$) plutôt que sur des métriques infrastructurelles.

3.3.5.0.4 H4 – Indépendance des intents. Chaque intent $e \in E$ est traité de manière indépendante, sans mémoire des décisions de routage antérieures et sans contexte conversationnel partagé entre requêtes. Cette hypothèse simplifie l’évaluation comparative en éliminant les effets de cache et les biais de séquence, au prix d’ignorer des optimisations potentielles telles que la mise en cache des réponses pour intents récurrents. L’extension à un routage avec mémoire constitue une perspective discutée au chapitre 6.

3.3.6 Qualité agrégée (AIQ)

L’évaluation du chapitre 5 repose sur quatre métriques, définies ici. Soit $\mathcal{T} \subset E$ le jeu d’évaluation, intents générés par LLM sur les quatre domaines (éq. (3.2)), annotés et revus par l’expertise réseau du projet (§5.1.1). La qualité moyenne d’inférence (*Average Inference Quality*, AIQ), suivant RouterBench [47], est :

$$\text{AIQ} = \frac{1}{|\mathcal{T}|} \sum_{e \in \mathcal{T}} q_{R^*(e)}(e), \quad (3.7)$$

où $|\mathcal{T}|$ désigne le cardinal du jeu d’évaluation et $q_{R^*(e)}(e) \in [0, 1]$ la qualité mesurée a posteriori par le LLM Juge sur la réponse du modèle sélectionné $R^*(e)$ (éq. (3.6)). Une AIQ élevée traduit une bonne adéquation du routeur à la difficulté des intents.

3.3. FORMULATION DU PROBLÈME

3.3.7 Latence bout en bout (P50, P99)

Soit $T(e)$ le temps total mesuré pour traiter un intent e , décomposé en :

$$T(e) = T_{\text{est}}(e) + T_{\text{rout}}(e) + T_{\text{inf}}(e) + T_{\text{juge}}(e), \quad (3.8)$$

où $T_{\text{est}}(e)$ est le temps d'estimation de complexité, $T_{\text{rout}}(e)$ le temps de décision du routeur, $T_{\text{inf}}(e)$ le temps d'inférence du modèle $R^*(e)$ et $T_{\text{juge}}(e)$ le temps d'évaluation par le LLM Juge.¹ Nous rapportons les percentiles $P_{50}(T)$ et $P_{99}(T)$ sur $\mathcal{T}$, conformément aux pratiques d'évaluation des systèmes en production [47]. La moyenne masquerait les pics de queue, qui déterminent le respect d'un SLA ; le P99 les expose. La contrainte opérationnelle est $P_{99}(T) \leq \max_{e \in \mathcal{T}} L_{\text{max}}(e)$.

3.3.8 Coût agrégé proxy

Pour chaque intent e , le coût d'inférence est le produit du tarif unitaire du modèle sélectionné $c_{R^*(e)}$ (en unités monétaires pour 1000 jetons, normalisé sur la grille tarifaire OpenRouter en vigueur au moment des expériences, cf. chapitre 5) par le nombre total tokens(e) de jetons d'entrée et de sortie consommés. Le coût agrégé sur le jeu d'évaluation $\mathcal{T}$ correspond à la moyenne arithmétique des coûts par intent. Cette définition reprend la métrique de coût moyen de RouterBench [47] en l'adaptant au cas où le tarif dépend du modèle sélectionné par le routeur et non d'un modèle unique.

3.3.9 Distribution de routage

On note π_i la proportion d'intents du jeu d'évaluation $\mathcal{T}$ routés vers le modèle $m_i \in M$. Le vecteur de distribution $(\pi_1, \pi_2, \dots, \pi_k)$ permet de vérifier l'absence de "collapse" du routeur sur un sous-ensemble restreint du pool. On rapporte également la corrélation croisée entre la décision du routeur $R^*(e)$ et le domaine $d(e)$ de l'intent, qui diagnostique l'effectivité de la spécialisation par domaine posée en hypothèse H1 (§3.3.5.0.1).

3.3.10 Critère principal de comparaison

Les quatre métriques précédentes alimentent un plan coût / qualité dans lequel sont projetées les stratégies comparées (ALWAYS-HEAVY, ALWAYS-LIGHT, RANDOM, InferRouter-LLM). ALWAYS-HEAVY route chaque intent vers le modèle le plus capable, et le plus coûteux, du pool ; ALWAYS-LIGHT vers le moins coûteux ; RANDOM tire le modèle uniformément au hasard dans M . Ces trois stratégies ignorent le contenu de l'intent et encadrent le gain attribuable au routage informé. La métrique principale d'évaluation est la position de chaque stratégie sur la frontière de Pareto qualité/coût sous contrainte de latence, telle que définie par Hu et al. [47] : une stratégie est dite Pareto-dominante si elle réalise simultanément une AIQ supérieure et un coût agrégé inférieur à une stratégie de référence, tout en satisfaisant la contrainte $P_{99}(T) \leq \max_{e \in \mathcal{T}} L_{\text{max}}(e)$. Cette projection synthétique sera l'objet central du chapitre 5.

¹La décomposition omet volontairement un terme de latence réseau $T_{\text{net}}(e)$. Le routeur étant co-localisé avec le contrôleur IBN sur un nœud de bordure, le transport intra-cluster reste de l'ordre de quelques millisecondes, négligeable devant $T_{\text{inf}}(e)$ qui dépasse typiquement 100 ms pour un LLM appelé en cloud.

Chapter 4

InferRouter-LLM

Ce chapitre détaille l’architecture d’InferRouter-LLM et ses trois contributions. Il prolonge la formulation du chapitre 3 côté implémentation et prépare l’évaluation du chapitre 5. Le système existe sous forme d’une chaîne complète et exécutable ; les choix exposés ici intègrent les enseignements des campagnes de mesure.

4.1 Vue d’ensemble de l’architecture

InferRouter-LLM s’organise autour d’un routeur central qui reçoit un intent réseau textuel et choisit le LLM cible vers lequel déléguer l’inférence. La chaîne de traitement enchaîne quatre étapes. L’estimateur de complexité analyse l’énoncé et en déduit un niveau de difficulté. Le routeur croise ce niveau avec les attributs de coût et de latence des candidats, restreint le pool aux modèles admissibles (éq. (3.5)), puis sélectionne le moins coûteux dont la qualité attendue $\hat{q}_i(e)$ atteint le plancher exigé par la criticité de l’intent (éq. (3.6)). Le LLM cible produit la réponse. Le LLM Juge local note cette réponse a posteriori, ce qui alimente la calibration des seuils hors ligne. La Figure 4.1 donne la vue d’ensemble de cette chaîne et de son déploiement.

Le routeur n’exécute aucune inférence générative. Cette séparation isole la logique de décision des modèles sous-jacents et autorise l’ajout ou le retrait d’un candidat sans toucher au cœur du routage. Le routeur et le Juge sont co-localisés avec le contrôleur IBN sur un nœud de bordure, ce qui rend le transport réseau de l’intent négligeable devant l’inférence cloud des modèles cibles (§3.1.1.0.2).

Le prototype matérialise cette chaîne en modules Python découplés : un schéma d’intent, un client OpenRouter pour les modèles cibles, un wrapper du Juge local via Ollama, l’estimateur de complexité et la politique de routage. Le pool testé oppose un modèle léger, c’est-à-dire de coût unitaire c_i et de latence faibles mais de capacité moindre, à un modèle lourd, de grande capacité mais plus coûteux et plus lent. Cette configuration volontairement minimale isole le gain du routage.

4.2 Budget de latence par étage

Le chapitre 1 fixe une contrainte de conception : l’estimateur doit répondre en moins de 15 ms (§1.3). Les mesures ci-dessous en constituent la première vérification chiffrée, étage par étage de la chaîne. L’estimateur, tel qu’implémenté initialement, rechargeait le modèle persisté à chaque appel : 31 ms

4.2. BUDGET DE LATENCE PAR ÉTAGE

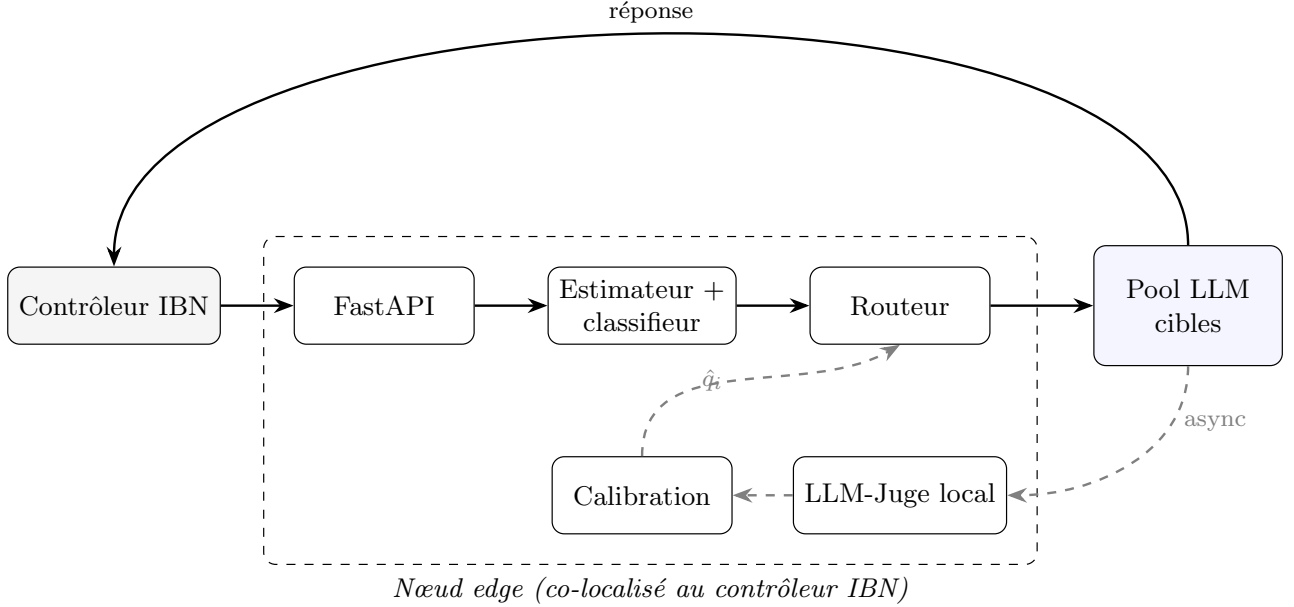


Figure 4.1: Architecture système d’InferRouter-LLM. Le chemin critique (traits pleins) enchaîne estimation, routage et inférence du modèle sélectionné $R^*(e)$, appelé en cloud. La co-localisation du routeur avec le contrôleur réseau rend le transport intra-cluster (≤ 5 ms) négligeable devant l’inférence cloud (plusieurs secondes). Le LLM-Juge local évalue les réponses hors chemin critique (traits pointillés) et met à jour la table de calibration $\hat{q}_i$ utilisée par le routeur.

mesurés, au-dessus du budget. Le modèle gardé en mémoire entre les appels, comme le ferait un service déployé, ramenait cette latence à 15 ms.

L’écart restant venait de la parallélisation du forêt aléatoire (`n_jobs=-1`). Elle est pertinente à l’entraînement sur 252 exemples, mais coûte plus cher qu’elle ne fait gagner sur une prédiction unitaire : le lancement des threads y dépasse le calcul qu’ils parallélisent. Figée à un seul processus pour l’inférence, elle ramène la latence à 3,4 ms. La décision de routage elle-même, purement calculatoire, s’exécute en 0,004 ms.

Table 4.1: Budget de latence mesuré par étage de la chaîne (P50).

Étage	Latence P50	Note
Estimateur de complexité	3,4 ms	modèle en mémoire, 1 processus
Décision de routage	0,004 ms	calcul pur, sans réseau
Transport réseau (edge)	négligeable	hypothèse ADR-004
Inférence, modèle léger	8,5 s	mesuré, 137 appels API
Inférence, modèle lourd	41,4 s	mesuré, 103 appels API

Le contraste est net : l’estimateur et la décision de routage représentent ensemble moins de 4 ms, contre plusieurs secondes à plusieurs dizaines de secondes pour l’inférence du modèle cible. Le coût du

4.3. POSITIONNEMENT DANS LA BOUCLE IBN ET PÉRIMÈTRE

routage lui-même est donc négligeable devant celui qu’il cherche à arbitrer ; le gain ou la perte de latence d’InferRouter-LLM face aux stratégies de référence (§5.4.1) se joue entièrement sur le choix du modèle cible, pas sur le coût de la décision.

4.3 Positionnement dans la boucle IBN et périmètre

La boucle IBN à cinq composantes (§2.1.1) situe précisément notre apport. InferRouter-LLM intervient en amont immédiat de la phase de *translation* : l’intent arrive en langage naturel, et le système décide quel modèle du pool doit le traiter. La sortie reste textuelle (recommandation, plan d’action, diagnostic), destinée à l’opérateur ou à l’orchestrateur aval. Résolution, activation et assurance restent hors périmètre : rien n’est poussé vers l’infrastructure.

Ce placement répond à un problème concret des IBNS qui intègrent des LLM : la soutenabilité économique à l’échelle opérateur. Un contrôleur reçoit un flux continu d’intents dont une large part sont des lectures d’état élémentaires. Les confier tous à un modèle frontière coûte sans bénéfice : nos mesures montrent qu’un modèle léger déployable en bordure suffit sur ces intents simples (§5.4.1). La couche de routage réserve le modèle lourd aux intents qui le justifient, sans rien changer au reste de la boucle.

L’exclusion de l’actuation est un choix motivé, pas une limite subie. Appliquer une sortie de LLM sur une infrastructure vive exige la validation formelle qu’assurent les phases de résolution et d’activation de la boucle classique ; produire des configurations exécutées sans ces garde-fous contredirait le principe d’autonomie supervisée énoncé par la RFC 9315 [5].

Notre évaluation du Juge va dans le même sens (§5.3.2) : un juge local fiable pour départager des candidats nettement différents ne l’est pas comme oracle absolu. Il suffit pour router, pas pour autoriser seul une action. La criticité $\kappa(e)$ portée par chaque intent prépare ce prolongement : dans une boucle fermée complète, elle conditionnerait le niveau de validation exigé avant toute exécution.

Cette position complète enfin les travaux d’IBN agentique, où un LLM pilote la boucle de bout en bout jusqu’à l’exécution [48]. Ces agents supposent résolu le choix du modèle qui les anime. Notre couche fournit cette politique de sélection coût-qualité, et reste utilisable aussi bien par un opérateur humain que par un tel agent.

4.4 Contribution 1 : estimateur de complexité sémantique

L’estimateur traduit les trois critères posés au chapitre 3 (§3.2) en attributs calculés sur l’énoncé : le nombre d’entités $n(e)$, la profondeur d’inférence $p(e)$ approximée par le nombre de contraintes (au sens fixé en §3.2.2), et le nombre de domaines croisés $|\delta(e)|$. Une tête `scikit-learn` (régression logistique ou forêt aléatoire) classe l’intent en trois niveaux, simple, medium ou complexe, en moins de 15 ms.

Nous fondons l’estimateur sur ces attributs, et non sur la proximité d’embeddings bruts, pour deux raisons : ils implémentent directement les critères du chapitre 3, ce qui rend chaque décision explicable en termes d’entités, de contraintes et de domaines, et ils sont insensibles par construction à la verbosité de l’énoncé. Parmi eux, la profondeur d’inférence, captée par le nombre de contraintes, porte l’essentiel du pouvoir prédictif une fois la longueur neutralisée. L’évaluation nuance toutefois ce choix : sur les 252 intents, des embeddings de phrase séparent la complexité mieux que les attributs, et la combinaison des deux signaux fait encore mieux (§5.2.3). Les attributs restent le socle du prototype ; l’estimateur

combiné est la piste du système final.

L'entraînement révèle un piège que nous documentons. Sur un jeu généré naturellement, la longueur de l'énoncé suffit à prédire la complexité, parce que les intents simples sont courts et les complexes longs. Ce confond produit une exactitude trompeuse. Mesuré à longueur contrôlée, le signal porté par les attributs se situe entre 65 et 73 %, bien au-dessus de la baseline de 33 %, mais loin du chiffre gonflé par la verbosité. Le chapitre 5 détaille cette mesure honnête.

4.5 Contribution 2 : LLM cibles spécialisés par domaine réseau

Le pool de modèles cibles mêle des LLM génériques, qui couvrent tous les domaines et servent de repli sur les intents multi-domaines, et des LLM spécialisés par domaine réseau (RAN, cœur, sécurité, slice). Le routeur arbitre selon la complexité estimée et les contraintes de l'intent. Un intent simple monodomaine peut être confié à un LLM spécialisé moins coûteux ; un intent multi-domaines exige un généraliste.

Le routeur sélectionne le candidat admissible le moins coûteux dont la qualité attendue $\hat{q}_i(e)$ atteint le plancher, les égalités de coût étant tranchées par la qualité puis par la latence (§3.3.4). Ce dernier point n'est pas un détail : un spécialiste de domaine partage le tarif de son modèle de base, si bien que sans ce départage son avantage mesuré ne serait jamais réclamé. La qualité attendue par candidat est estimée a priori, puis calibrée par les mesures a posteriori $q_i(e)$ du Juge.

L'évaluation expose une condition de validité de cette contribution. Le gain du routage suppose qu'un modèle léger devienne compétitif sur les intents simples. Sur le couple testé (**llama-3.2-3b** contre **claude-sonnet-4.6**), le léger reste en dessous partout, y compris sur le simple, ce qui fait dégénérer le routage en stratégie ALWAYS-HEAVY (§3.3.10). Un léger plus fort (**gpt-4o-mini**) réduit l'écart sans le refermer sur cet échantillon. Le choix du pool devient donc un critère de conception à part entière, point développé en §5.4.

4.6 Contribution 3 : LLM Juge (RocketEval)

Le LLM Juge évalue la qualité d'une réponse par la méthode RocketEval [12], exécutée localement via Ollama. La méthode opère en deux temps. Un modèle fort génère hors ligne une checklist propre à chaque intent, soit une liste de critères vérifiables Y/N. Le juge local grade ensuite chaque item sur la réponse candidate, puis les notes sont agrégées en un score dans $[0, 1]$. La checklist par intent est le levier central : une checklist fixe identique pour tous les intents échoue, notamment sur l'aveuglement à l'hallucination (§5.3).

L'exécution locale tient à deux raisons. La latence d'évaluation reste maîtrisée et constante. Les intents réseau, potentiellement sensibles, ne quittent pas le périmètre de l'opérateur.

L'évaluation circonscrit précisément la portée de ce juge. En discrimination grossière, départager un bon candidat d'un mauvais, un juge local **gemma2:9b** doté de RocketEval est fiable. En discrimination fine, détecter une seule erreur injectée dans une réponse sinon correcte, aucun juge local testé ne convient, et ni la taille ni le mode de notation ne corrigent ce point. Le routage ne demande que la discrimination grossière. Nous employons donc le juge comme signal de routage en comparaison relative entre candidats, et non comme oracle d'une qualité absolue fine. La contribution C3 tient pour cet usage, qui est celui qu'elle revendique. La fiabilisation du juge pour un usage plus fin est

laissée à des travaux ultérieurs.

4.7 Auto-calibration des seuils

Le Juge sert aussi à calibrer les seuils du routeur hors ligne. La procédure mesure la qualité a posteriori $q_i(e)$ de chaque famille de modèles par niveau de complexité, puis ajuste l'estimation a priori $\hat{q}_i(e)$ qui guide la sélection (éq. (3.6)). Cette boucle relie le Juge et l'estimateur sans intervention humaine au runtime, conformément à l'hypothèse H2 (§3.3.5).

La calibration s'appuie sur l'ordre relatif fourni par le juge, robuste, plutôt que sur ses valeurs absolues, qui se dégradent sur les intents complexes (§5.4). Tant que le juge reste fiable en comparaison grossière, la calibration garde son sens. Sa fiabilisation pour les cas fins reste un prérequis à une calibration plus précise, balisée dans le travail futur du chapitre 5.

Chapter 5

Évaluation

Ce chapitre rend compte d’une campagne préliminaire de validation des risques, menée avant la construction du système complet. Le but n’était pas de produire un benchmark final, mais de tester les hypothèses dont dépend toute la chaîne : la fiabilité du LLM Juge, le pouvoir prédictif de la complexité sémantique, et la prémisse économique du routage. Plusieurs résultats sont négatifs. Nous les assumons comme tels, car ils ont chacun écarté une voie coûteuse avant qu’elle ne le devienne.

5.1 Protocole expérimental

5.1.1 Jeu de données d’intents

L’évaluation s’appuie sur un jeu de 252 intents réseau générés par LLM, réparti sur les quatre domaines fonctionnels de l’équation (3.2) (RAN, cœur, sécurité, slice) et trois niveaux de complexité (simple, medium, complex). La génération par LLM remplace le pipeline IBNs de Saha [49] : ce dernier produit des énoncés de flux Software Defined Networking (SDN) bas niveau, sans domaine telco ni annotation de complexité, inadaptés à notre architecture. Chaque intent porte six attributs, récapitulés dans la Table 5.1 avec leurs valeurs possibles et leur méthode d’attribution.

Table 5.1: Attributs d’un intent du jeu de données : valeurs possibles et méthode d’attribution.

Attribut	Valeurs possibles	Méthode d’attribution
id	slug court unique	produit à la génération
text	énoncé en langage naturel (anglais)	généré par LLM, déduplicué, revu
domain ($d(e)$)	ran, core, security, slice	fixé par la cellule de génération, revu
expected_complexity	simple, medium, complex	fixé par la cellule selon la grille de la Table 5.2, revu
criticality ($\kappa(e)$)	low, med, high	proposé par le générateur, revu
slice_type	embb, urllc, mmhc, null	proposé par le générateur, revu

L’annotation de complexité est reliée aux trois critères du chapitre 3 par la grille de la Table 5.2 :

5.1. PROTOCOLE EXPÉRIMENTAL

chaque niveau est défini par des bornes sur le nombre d’entités $n(e)$, la profondeur d’inférence $p(e)$ et le nombre de domaines croisés $|\delta(e)|$. Cette grille est imposée au générateur comme consigne, puis vérifiée en revue. Elle exige que le gradient de complexité soit porté par le contenu technique, jamais par la longueur de l’énoncé ; les attributs calculés de la §5.2.2 en sont des approximations mesurées ensuite automatiquement.

Table 5.2: Grille d’annotation de la complexité, exprimée sur les trois critères du chapitre 3. Au niveau complexe, le croisement de domaines est majoritaire mais non systématique.

Niveau	$n(e)$	$p(e)$	$ \delta(e) $	Profil type
simple	1–2	1	1	lecture d’état ou requête directe, sans optimisation
medium	modéré	2	1	diagnostic guidé ou reconfiguration ciblée, une contrainte
complexe	élevé	≥ 3	≥ 2	arbitrage multi-contraintes, plusieurs SLA simultanés

La génération procède par cellule de la matrice domaine $\times$ complexité : un appel au modèle générateur (`claude-sonnet-4.6` via OpenRouter, température 0,7) produit un lot de 21 intents par couple, soit 12 cellules et 252 intents.

Le prompt de génération fixe le domaine et le niveau de complexité de la cellule, rappelle la grille de la Table 5.2, exige l’ancrage dans des scénarios 3GPP, O-RAN et ETSI ZSM avec des identifiants plausibles, et interdit les quasi-doublons. Trois intents de référence, tirés d’une amorce de 20 intents écrits à la main, servent d’exemples de style et de format.

Trois filtres automatiques s’appliquent à chaque lot. La validation de schéma écarte et journalise toute entrée malformée. Le domaine et la complexité sont forcés aux valeurs de la cellule, ce qui interdit toute dérive d’étiquette par le générateur. Une déduplication par texte normalisé élimine les doublons. Les journaux de la campagne de génération initiale n’ont pas été conservés, si bien que son taux de rejet n’est pas reconstituable. Une mesure de contrôle a posteriori, même invite et même modèle, sur quatre lots de 21 intents couvrant trois domaines et trois niveaux de complexité, donne 84 entrées produites pour 84 retenues : aucune entrée malformée, aucun doublon. Ces filtres jouent donc un rôle de garde-fou plutôt que de tri effectif, au moins avec le modèle de génération retenu.

Chaque intent est ensuite revu manuellement par l’auteur, selon trois conventions arrêtées pendant la revue : la criticité mesure l’impact de l’action (une lecture de KPI ne dépasse pas med, hors lectures de sécurité) ; au niveau complexe, le domaine désigne le point d’entrée de l’intent, pas l’unique champ mobilisé ; `slice_type` vaut null pour les intents multi-slices ou agnostiques de slice.

Cette revue a corrigé treize annotations (dix criticités sur-cotées, deux domaines, une complexité) sans écarter d’intent : le total reste 252, avec une matrice domaine $\times$ complexité quasi équilibrée (19 à 24 intents par cellule).

Un biais de génération est apparu dès les premiers entraînements : dans un jeu produit “naturellement” (variante v1), les intents simples sont courts et les intents complexes sont longs. La longueur seule devient alors un prédicteur quasi parfait, sans rapport avec le fond sémantique. Pour mesurer honnêtement le signal de complexité, nous avons généré deux variantes contrôlées. La variante v2

5.2. ESTIMATEUR DE COMPLEXITÉ SÉMANTIQUE

fixe une longueur quasi constante entre classes (simple 42, medium 41, complex 47 mots en moyenne). La variante v3 décorrèle la longueur de la complexité par tirage aléatoire d’une cible de longueur par intent, ce qui produit des plages chevauchantes (simple de 4 à 54 mots, complex de 9 à 126). La variante v1 reste le jeu naturel destiné au routeur et aux benchmarks ; v2 et v3 servent à mesurer la validité de l’estimateur.

5.1.2 Modèles, juge et métriques

Le pool de modèles cibles oppose un modèle léger à un modèle lourd, tous deux appelés via OpenRouter. Le léger initial est `llama-3.2-3b-instruct`, remplacé ensuite par `gpt-4o-mini` pour tester un candidat plus compétitif. Le lourd est `claude-sonnet-4.6`. Le LLM Juge s’exécute localement via Ollama, en `gemma2:2b` puis `gemma2:9b`, suivant la méthode RocketEval [12]. La Table 5.3 donne la taille et le tarif unitaire c_i de chaque modèle : près de deux ordres de grandeur séparent le léger initial du lourd. Les métriques sont celles du chapitre 3 : qualité agrégée (équ. (3.7)), latence P_{50} et P_{99} (équ. (3.8)), coût agrégé et distribution de routage.

Table 5.3: Modèles de la campagne : rôle, taille et tarif unitaire c_i en USD pour 1000 jetons (grille OpenRouter en vigueur au moment des expériences). n.c. : taille non communiquée par l’éditeur ; n.d. : tarif non consigné. Les juges locaux s’exécutent via Ollama, sans coût d’appel.

Modèle	Rôle	Taille (Mds)	c_i entrée	c_i sortie
<code>llama-3.2-3b-instruct</code>	cible légère initiale	3	0,000051	0,000335
<code>gpt-4o-mini</code>	cible légère (seconde)	n.c.	0,00015	0,0006
<code>qwen-2.5-72b</code>	cible légère retenue	72	0,00036	0,0004
<code>claude-sonnet-4.6</code>	génération du dataset et des checklists	n.c.	0,003	0,015
<code>claude-opus-4.8</code>	cible lourde retenue ; jugement de référence	n.c.	0,005	0,025
<code>gemma2:2b</code>	juge local	2	exécution locale	
<code>gemma2:9b</code>	juge local	9	exécution locale	

Le jugement de référence est produit en aveugle par un modèle fort (Claude Opus), sur des réponses anonymisées et mélangées. Cette calibration petit-juge contre juge-fort est une pratique standard [43, 40], mais un jeu validé par un expert humain resterait souhaitable. Les expériences sur l’estimateur utilisent une validation croisée à cinq plis ($k = 5$). Le coût des appels payants a été suivi tout au long : la campagne live a consommé environ 12\$ sur OpenRouter, ce qui a borné la taille des échantillons et arrêté certains runs avant terme.

5.2 Estimateur de complexité sémantique

5.2.1 Un confond de longueur trompeur

Entraîné sur la variante naturelle v1, l’estimateur atteint 99,6 % d’exactitude en validation croisée. Le chiffre est un leurre : la seule variable `n_tokens` (nombre de jetons de l’énoncé) atteint elle aussi

5.2. ESTIMATEUR DE COMPLEXITÉ SÉMANTIQUE

99,6 %, car la complexité annotée est presque parfaitement corrélée à la longueur dans le jeu naturel. Nous écartons ce résultat : le modèle ne lit que la verbosité.

5.2.2 Mesure honnête à longueur contrôlée

L’extracteur d’attributs calcule six mesures par énoncé, par heuristiques lexicales sans modèle : **n_entities** (entités réseau distinctes, proxy de $n(e)$), **n_constraints** (marqueurs de contrainte et seuils SLA chiffrés, proxy de $p(e)$), **n_domains** (domaines fonctionnels évoqués, proxy de $|\delta(e)|$), **n_numbers** (valeurs numériques porteuses d’unité), **n_tokens** (mots) et **n_sentences** (phrases). Nous appelons attributs de fond les quatre premiers, qui ne portent aucune mesure de longueur.

Une fois la longueur neutralisée, le tableau change. Sur la variante v2, la longueur seule chute à 45,2 %, contre 99,6 % en v1 : le confond est cassé. Les attributs de fond atteignent 67,1 % ; un RandomForest sur l’ensemble des attributs monte à 73,0 %, pour une baseline majoritaire de 33,3 % (classes équilibrées). Le signal sémantique existe donc, au-delà du hasard. La Table 5.4 récapitule ces mesures, y compris pour les embeddings de phrase discutés en §5.2.3.

Table 5.4: Exactitude de l’estimateur de complexité par variante du jeu de données et par signal (validation croisée $k = 5$ sur 252 intents, régression logistique sauf mention RF, baseline majoritaire 33,3 %).

Signal	v1 (naturel)	v2 (long. contrôl.)	v3 (décorrélé)
Longueur seule	99,6 %	45,2 %	48,8 %
Attributs de fond	79,8 %	67,1 %	64,7 %
Tous attributs (RF)	99,2 %	73,0 %	73,0 %
Embeddings seuls	96,0 %	93,7 %	84,9 %
Attributs + embeddings	97,6 %	94,4 %	85,3 %

Parmi les attributs de fond, le nombre de contraintes (**n_constraints**), que nous utilisons comme proxy de la profondeur d’inférence $p(e)$ (§3.2.2), domine après contrôle de la longueur. Les seules entités et domaines plafonnent autour de 50 % ; l’ajout des contraintes les fait passer à 63 %. La profondeur de raisonnement, et non le nombre d’entités, porte l’essentiel du pouvoir prédictif.

La variante v3 généralise mieux. Un estimateur entraîné sur v3 classe correctement quatre intents courts naturels sur cinq, contre deux sur cinq pour un estimateur entraîné sur v2. Nous retenons v3 comme jeu de référence de l’estimateur, parce que la décorrélation longueur-complexité le force à apprendre le fond plutôt que la verbosité. Le chiffre à retenir n’est pas le 99,6 % initial, mais les mesures obtenues sans confond : 65 à 73 % sur les attributs, jusqu’à 85 % en y joignant les embeddings.

5.2.3 Embeddings de phrase : un signal réévalué

Un premier test, mené sur les 20 intents de l’amorce manuelle (**all-MiniLM-L6-v2** suivi d’un k-NN en leave-one-out), prédisait la complexité à 35 %, soit le niveau de la baseline. Nous en avons conclu que l’espace sémantique encodait le sujet de l’intent, pas sa difficulté, et ce constat a orienté l’estimateur vers les attributs calculés.

Réévalué sur les 252 intents avec une régression logistique, ce verdict ne tient pas. Les embeddings seuls atteignent 93,7 % sur v2 et 84,9 % sur v3, au-dessus des attributs (73,0 %) ; la combinaison des

deux signaux monte à 94,4 et 85,3 % (Table 5.4). La longueur n’explique pas ce résultat, puisqu’elle ne prédit que 48,8 % sur v3. Les mêmes embeddings prédisent aussi le domaine fonctionnel à 81,7 % (baseline 25,8 %). Le test initial concluait au hasard par manque d’échantillon et par faiblesse du classifieur, pas par absence de signal.

Ce constat ne renverse pas le choix du prototype, il le borne. Les attributs calculés restent lisibles en termes métier (entités, contraintes, domaines), implémentent directement les critères du chapitre 3 et sont insensibles à la verbosité par construction ; le modèle embarqué dans le routeur s’appuie sur eux. Pour le système final, l’estimateur combiné attributs + embeddings devient la référence à battre.

5.3 Fiabilité du LLM Juge

5.3.1 Checklist fixe contre RocketEval complet

Le premier juge note chaque réponse sur une checklist fixe de quatre critères binaires (oui/non), identiques pour tous les intents : correction technique au regard de l’intent, complétude de la réponse, adéquation au domaine réseau, absence de faits ou de valeurs inventés. Le score est la proportion de critères satisfaits. Ce juge échoue. Sur 20 intents, son accord avec le jugement de référence est de 40 % en **gemma2:2b**, sous le seuil de 60 % que nous nous étions fixé. Passer à **gemma2:9b** ne suffit pas : l’accord monte à 50 %, toujours sous le seuil. Trois défauts reviennent : le juge récompense les hallucinations (note de 1,00 à une réponse qui invente un nombre d’User Equipment (UE) enregistrés), classe parfois la pire réponse au-dessus de la meilleure, et sature autour de 0,88 à 1,00 sur les intents complexes sans discriminer. Le levier n’est pas la taille du juge, c’est la méthode d’évaluation.

Nous remplaçons la checklist fixe par RocketEval complet [12] : une checklist générée par intent via un modèle fort (**claude-sonnet-4.6**), gradée ensuite par le même petit juge local. Un exemple de checklist générée figure en annexe A.3. L’accord passe à 90 % (18 sur 20) en **gemma2:2b**, et l’aveuglement à l’hallucination est corrigé : une réponse qui invente un chiffre tombe de 1,00 à 0,43, et l’ordre correct est rétabli. Un juge de deux milliards de paramètres doté d’une bonne checklist dépasse donc largement un juge de neuf milliards à checklist fixe : la méthode prime sur la taille.

5.3.2 Discrimination grossière contre discrimination fine

Le 90 % précédent est optimiste. Il oppose un modèle fort à un modèle faible, ce qui produit une référence sans variance : le lourd l’emporte sur les vingt intents, et un juge qui dirait toujours “lourd” obtiendrait 100 %. La métrique d’accord est donc dégénérée. Pour la corriger, nous construisons un test décisif : pour chaque intent, nous dégradons la bonne réponse en y injectant une seule erreur (chiffre faux, étape retirée, affirmation fausse), puis nous demandons au juge de classer l’originale et la dégradée. La vérité-terrain est connue et les deux textes sont quasi identiques.

Le juge échoue sur cette tâche fine. En **gemma2:2b**, la préférence correcte tombe à 35 %, avec 40 % d’égalités et 25 % d’inversions. Par type d’erreur, le chiffre faux est détecté à 57 %, l’affirmation fausse à 33 %, mais l’étape manquante à 14 % seulement, avec quatre inversions : retirer une étape critique fait souvent monter le score de la version dégradée, parce que le texte paraît plus propre. Grossir le juge n’aide pas : en **gemma2:9b**, la préférence correcte chute à 5 % avec 90 % d’égalités. La notation par paires, qui ne dépend pas de la checklist, descend à 0 % de préférence correcte et 100 % d’égalités : le juge répond sincèrement “égalité”.

5.4. PRÉMISSE DU ROUTAGE ET CALIBRATION

Deux régimes se distinguent nettement, résumés dans la Table 5.5. En discrimination grossière, un candidat nettement meilleur qu’un autre, le juge local est fiable : **gemma2:9b** avec RocketEval atteint 100 %. En discrimination fine, détecter une erreur isolée, aucun juge local testé ne convient. Ni la méthode de notation ni la taille du modèle ne corrigent ce point. La cause est le discernement du juge léger lui-même.

Table 5.5: Accord du LLM Juge selon le régime de discrimination et la configuration. Grossier : bon candidat contre mauvais. Fin : réponse correcte contre variante à une erreur injectée.

Configuration	Grossier	Fin (préf. correcte)
Checklist fixe, gemma2:2b	40 %	n/a
Checklist fixe, gemma2:9b	50 %	n/a
RocketEval, gemma2:2b	90 %	35 %
RocketEval, gemma2:9b	100 %	5 %
RocketEval pairwise, gemma2:9b	n/a	0 %

La conséquence pour le système est directe. Le routage ne demande que la discrimination grossière : pour un intent donné, le LLM spécialisé est-il nettement meilleur que le modèle générique. Un juge local **gemma2:9b** avec RocketEval répond à cette question. Nous l’employons donc comme signal de routage en comparaison relative entre candidats, et non comme oracle de qualité absolue sur lequel poser un seuil. Cet usage est suffisant pour la contribution C3, et c’est ce qu’elle revendique.

5.4 Prémisses du routage et calibration

5.4.1 Matrice de qualité par tier et complexité

Le routage par qualité n’économise que si un modèle léger devient compétitif sur les intents simples. Nous mesurons donc la qualité réelle, notée par le juge, de chaque tier sur chaque niveau de complexité. Avec **llama-3.2-3b** face à **claude-sonnet-4.6**, le modèle léger est inadéquat partout, y compris sur le simple : qualité 0,11 contre 0,55 pour le lourd, soit un écart de 0,45. La Table 5.6 détaille la matrice. L’écart se maintient sur le medium (0,13 contre 0,50) et le complex (0,03 contre 0,22).

Table 5.6: Qualité moyenne notée par le juge, par tier de modèle et niveau de complexité (échantillon de calibration, 12 intents équilibrés).

Complexité	llama-3.2-3b	gpt-4o-mini	claude-sonnet-4.6
simple	0,11	0,24	0,55
medium	0,13	0,30	0,50
complex	0,03	0,06	0,22

Le routage par “argmax q ” s’effondre alors en always-heavy : le léger ne suffit jamais, donc aucune économie de coût n’est possible sur ce couple. Ce résultat est instructif plutôt que décevant. Le gain du routage est conditionné par le choix du pool. Un modèle léger trop faible l’annule, ce qui fait du choix du pool un critère de conception à part entière.

5.4.2 Un léger plus compétitif

Nous testons un léger plus fort, **gpt-4o-mini**, en réutilisant le lourd et les checklists déjà calculés. Il double **llama** sur le simple (0,24 contre 0,11) et dépasse même le lourd sur un intent simple isolé (0,71 contre 0,57). En moyenne, il reste sous le lourd, avec un écart de 0,31 sur le simple contre 0,45 pour **llama**. La parité n'est pas démontrée sur cet échantillon. Un meilleur léger réduit l'écart, sans le refermer ici.

Une grande calibration de **gpt-4o-mini** face au lourd a été interrompue à 45 intents sur 72, crédits OpenRouter épuisés. Un défaut d'ordonnancement de l'échantillonneur a traité les classes complex puis medium avant simple : la ligne simple, pourtant la plus informative, n'a pas été recalculée à grande échelle. Le run reste donc partiel.

Le facteur limitant reste le juge local. Sévère et bruité sur le complexe, il n'accorde que 0,22 au lourd, valeur anormalement basse pour **claude-sonnet**, et ne discrimine pas sur ce régime, ce qui rejoint le 35 % de discrimination fine de la §5.3.2. Seul l'ordre relatif (le lourd au-dessus du léger, surtout sur le simple) est robuste ; les valeurs absolues sont peu fiables.

5.4.3 Parité atteinte avec un léger plus capable

La calibration précédente portait sur un échantillon déséquilibré, sans intent simple recalculé à grande échelle. Nous corrigeons l'échantillonneur (alternance des complexités plutôt qu'un traitement classe par classe, qui épuisait le budget avant d'atteindre les simples) et testons deux légers de capacité croissante.

qwen2.5:14b-instruct, exécuté localement via Ollama (coût nul), ferme l'écart sur le simple : 0,68 contre 0,60 pour le lourd, sur 26 intents. Le gradient des quatre légers testés est net : **llama-3.2-3b** (écart +0,45), **gpt-4o-mini** (+0,31), **qwen2.5:7b** (+0,20), **qwen2.5:14b** (−0,08). Plus le léger est capable, plus l'écart se referme, jusqu'à s'inverser.

qwen2.5:14b-instruct n'étant pas hébergé sur OpenRouter, sa latence ne se mesure qu'en conditions de développement, non représentatives d'un service de production. Nous recalibrons sur **qwen2.5-72b-instruct**, disponible en API, sur les 74 mêmes intents : parité confirmée (simple −0,04, medium +0,10, complex −0,07). Ce modèle devient la référence retenue pour le benchmark comparatif qui suit, avec latence et coût mesurés en conditions réelles.

La prémisses du routage, en échec sur le premier couple testé (§5.4.1), est donc levée : il existe un léger suffisamment capable pour égaler **claude-sonnet-4.6** sur les intents simples. Mais cette parité soulève aussitôt une question sur le choix du lourd, examinée ci-dessous.

5.4.4 Le choix du modèle lourd conditionne le routage

Un léger qui égale le lourd partout ruinerait le routage : autant tout router vers le léger. La valeur du routage ne vient pas de la parité mais du différentiel de qualité entre les tiers, et de sa variation avec la complexité. La quasi-parité globale mesurée contre **claude-sonnet-4.6** suggère donc que ce modèle est un plafond de référence trop bas.

Nous testons la robustesse du choix en confrontant le même léger à un lourd plus fort, **claude-opus-4.8**, sur les 74 intents, checklists inchangées. L'écart lourd–léger devient réel et croît avec la complexité : +0,30 sur le simple, +0,47 sur le medium, +0,52 sur le complexe. C'est exactement le profil qui donne sa valeur au routage : l'écart est le plus faible là où l'on peut se permettre le léger

5.5. BENCHMARK COMPARATIF DES QUATRE STRATÉGIES

(le simple) et maximal là où le lourd s'impose (le complexe). Une lecture qualitative de réponses appariées confirme que cet écart est de fond : sur un intent complexe, `claude-opus-4.8` couvre des exigences techniques précises (procédures 3GPP, RFC de révocation de jeton, borne de latence) que `claude-sonnet-4.6` manque malgré une réponse trois fois plus longue. Nous retenons donc `claude-opus-4.8` comme lourd du pool.

5.4.5 Du “meilleur modèle” au moindre coût sous plancher

Avec un lourd nettement supérieur, la règle initiale “choisir la plus haute qualité attendue” sélectionne toujours le lourd : le routage dégénère en ALWAYS-HEAVY. Le “succès” de routage obtenu contre `claude-sonnet-4.6` tenait en réalité à la faiblesse de ce lourd, seul cas où le léger pouvait le dépasser sur le simple.

Nous adoptons donc la formulation duale du chapitre 3 (éq. (3.6)) : minimiser le coût sous un plancher de qualité $q_{\min}$, lui-même fonction de la criticité de l'intent. Le routage s'exerce alors sur deux axes. La complexité fixe la qualité attendue de chaque modèle (le léger se dégrade quand elle monte) ; la criticité fixe le plancher exigé (`low` 0,35, `med` 0,50, `high` 0,70). Le routeur retient le modèle le moins cher qui dépasse ce plancher. Le léger l'emporte dans le coin “simple et peu critique”, le lourd partout où la complexité ou la criticité l'imposent.

5.5 Benchmark comparatif des quatre stratégies

Ce comparatif isole l'arbitrage entre deux tiers : le pool oppose `qwen2.5-72b-instruct` (léger) à `claude-opus-4.8` (lourd), sans les spécialistes de domaine, dont l'apport propre est étudié séparément (§5.7). Nous comparons quatre stratégies sur les 74 intents, avec les mêmes checklists neutres pour toutes : ALWAYS-HEAVY, ALWAYS-LIGHT, RANDOM (tirage uniforme seedé), et InferRouter-LLM sous la règle du moindre coût sous plancher.

Table 5.7: Comparatif des quatre stratégies (74 intents, checklists homogènes, AIQ = qualité moyenne notée par le juge, coût = prix API réel moyen par intent).

Stratégie	AIQ	Coût \$/intent	P50	P99
Always-Heavy	0,88	0,0285	18,9 s	34,9 s
Always-Light	0,46	0,0002	7,9 s	48,3 s
Random	0,65	0,0124	11,7 s	37,3 s
InferRouter-LLM	0,78	0,0201	19,5 s	41,8 s

InferRouter-LLM route 30 % des intents vers le léger et 70 % vers le lourd : un partage réel, pas une dégénérescence. Il se place entre les deux extrêmes, avec un arbitrage assumé : une qualité de 0,78 (contre 0,88 pour tout-Opus et 0,46 pour tout-léger) pour une réduction de coût réel de 30 % face à ALWAYS-HEAVY. Le point important est la comparaison à RANDOM : InferRouter-LLM gagne nettement en qualité (0,78 contre 0,65) tout en coûtant plus, parce qu'il concentre les appels au lourd là où le léger flanche (les intents complexes), là où le tirage aléatoire mélange les tiers à l'aveugle. Ce dernier point se lit pleinement au §5.6, où la supériorité sur RANDOM s'avère conditionnée au profil du léger.

5.5. BENCHMARK COMPARATIF DES QUATRE STRATÉGIES

Le message a changé de nature depuis les premières mesures. Le routage n'est pas "la même qualité pour moins cher" ; c'est un compromis coût-qualité explicite, où l'opérateur échange une baisse de qualité bornée contre une économie substantielle, en concentrant cette baisse sur les intents où elle coûte le moins.

5.5.1 Frontière de Pareto coût-qualité

Le plancher $q_{\min}$ est le paramètre de contrôle de ce compromis. En le faisant varier uniformément de 0 à 1, on parcourt tout le spectre entre ALWAYS-LIGHT et ALWAYS-HEAVY : c'est la frontière de Pareto coût-qualité du système (Figure 5.1). Le calcul réutilise les qualités réelles déjà mesurées au benchmark, sans nouvel appel.

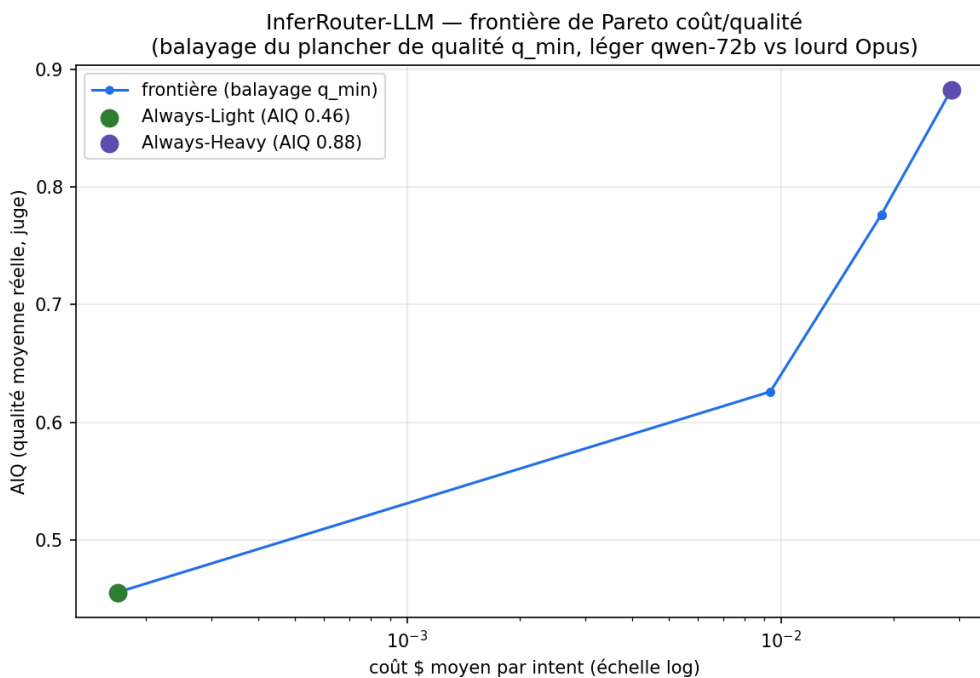


Figure 5.1: Frontière de Pareto coût-qualité obtenue en balayant le plancher $q_{\min}$, du tout-léger (bas gauche) au tout-lourd (haut droite).

La courbe présente trois paliers : le pool ne comptant que deux modèles et la décision ne dépendant que des trois niveaux de complexité, le routeur n'a que trois points de fonctionnement. Un pool plus riche (modèle intermédiaire, estimateur continu) la lisserait. Le point de bascule est net : vers $q_{\min} \approx 0,40$, le routeur envoie les intents simples au léger, la qualité passe de 0,46 à 0,78 pour un coût réduit d'un tiers. L'opérateur choisit son point de fonctionnement sur cette courbe selon son exigence de qualité.

5.6 Le choix du léger : le meilleur modèle n'est pas le bon

Le modèle léger retenu, **qwen2.5-72b-instruct**, n'est pas le meilleur candidat disponible. Nous avons comparé six modèles sur les 74 intents, avec le même juge et les mêmes checklists neutres, en mesurant la qualité réelle et le coût API réel par appel (Table 5.8).

Table 5.8: Six modèles comparés comme léger (qualité par complexité, coût API réel moyen par appel). Le coût réel, et non le tarif par jeton, tranche : **qwen3.5-flash**, le moins cher au jeton, est le plus cher à l'usage par verbosité.

Modèle	simple	medium	complex	coût \$/appel
gemini-2.5-flash-lite	0,42	0,39	0,36	0,00011
qwen-2.5-72b	0,64	0,39	0,32	0,00017
qwen3.5-flash	0,66	0,43	0,41	0,00271
gemini-2.5-flash	0,54	0,42	0,53	0,00101
deepseek-v3.2	0,53	0,52	0,58	0,00013
claude-opus-4.8 (lourd)	0,94	0,86	0,84	0,02850

Sur le rapport qualité/coût réel, **deepseek-v3.2** domine : qualité moyenne la plus élevée des légers (0,54), coût quasi le plus bas, et un profil plat qui ne s'effondre pas sur le complexe. Il surpasse **qwen-2.5-72b** sur les deux axes. Le remplacer par lui semble évident.

Il n'en est rien, et c'est le résultat le plus contre-intuitif de ce travail. Nous avons rejoué le benchmark des quatre stratégies avec chaque léger, à partir des mesures déjà acquises (aucun appel supplémentaire).

Table 5.9: Benchmark selon le léger du pool. Un léger plus fort dans l'absolu dégrade la valeur du routage.

Léger	InferRouter (AIQ)	Random (AIQ)	Verdict
qwen-2.5-72b	0,78	0,65	le routage bat le hasard
deepseek-v3.2	0,67	0,71	le routage perd contre le hasard

Avec **deepseek-v3.2**, InferRouter-LLM fait moins bien qu'un tirage aléatoire. La cause tient au profil plat du modèle. Le gain du lourd sur le léger n'y est pas uniforme : il vaut +0,42 sur les intents medium, mais seulement +0,27 sur les intents à haute criticité. Or le routeur envoie justement les intents critiques vers le lourd, là où le gain est le plus faible, et garde en léger les medium, là où le lourd aurait le plus aidé. Aucun de nos deux axes, complexité ou criticité, ne prédit où le lourd apporte le plus, parce que la faiblesse d'un léger plat ne suit aucun signal estimable. Sans signal, le routage informé ne peut pas battre le hasard.

Avec **qwen-2.5-72b**, au contraire, la qualité du léger décroît avec la complexité (0,64 à 0,32). Sa faiblesse est prédictible : elle suit la complexité, que l'estimateur sait estimer. Router les intents complexes vers le lourd cible alors exactement les intents où le léger flanche, et InferRouter-LLM bat le hasard.

Le critère de sélection d'un léger pour un routeur par complexité n'est donc pas sa qualité absolue, mais la corrélation entre sa faiblesse et un signal estimable. Un léger uniformément robuste, même supérieur, annule le bénéfice du routage. Ce constat nuance une hypothèse implicite de la littérature

du routage, qui traite le modèle économique comme interchangeable pourvu qu'il soit "assez bon" : sa *structure* de dégradation compte autant que son niveau.

5.6.1 Sensibilité au jeu d'intents et au pool

Le gain économique rapporté ci-dessus n'est pas une propriété du routeur seul : il dépend de ce qu'on lui donne à router. Nous le vérifions en rejouant le comparatif sur un échantillon distinct de 39 intents, tiré du même jeu.

Sur cet échantillon, l'économie face à ALWAYS-HEAVY tombe à 11 %, contre 30 % précédemment, à qualité inchangée (0,78). La raison tient entièrement à la répartition : neuf intents sur trente-neuf partent au tier économique, et tous appartiennent à la strate simple. Aucun intent moyen ou complexe n'y descend, le plancher de qualité que leur criticité impose excédant la qualité attendue du léger sur ces strates. Cinq intents simples restent eux-mêmes au modèle fort, leur criticité élevant le plancher au-delà de ce que le léger garantit.

L'économie mesurée reflète donc la proportion d'intents à la fois simples et peu critiques dans le flux traité. C'est le comportement attendu d'une politique indexée sur la criticité, non une instabilité : un opérateur dont le trafic d'administration est dominé par des consultations d'état économisera davantage qu'un opérateur dont chaque intent engage un service critique. Les taux annoncés doivent se lire avec cette réserve.

5.6.1.0.1 Effet du pool spécialisé. Le même échantillon, rejoué avec les quatre spécialistes de domaine ajoutés au pool, laisse la qualité d'InferRouter-LLM inchangée à 0,78, pour un coût voisin. Le routeur y délègue pourtant trente intents sur trente-neuf à un spécialiste et n'appelle jamais le modèle lourd générique, le départage à coût égal jouant systématiquement en faveur du spécialiste du domaine. Le gain de 0,038 mesuré en isolation (§5.7) ne ressort donc pas de l'Average Inference Quality (AIQ) agrégé : avec sept critères par checklist, le pas de mesure du juge vaut 0,14, soit près de quatre fois ce gain, qui se situe sous sa résolution.

Ce même ajout relève en revanche nettement RANDOM, de 0,62 à 0,76. L'effet est un artefact de composition : un tirage uniforme dans un pool comptant cinq modèles forts pour un seul économique se rapproche mécaniquement d'ALWAYS-HEAVY. Cette référence perd son sens dès que le pool est déséquilibré, ce qui justifie de la mesurer sur le pool à deux tiers.

5.6.2 Positionnement face aux routers publiés

Les routers publiés rapportent des réductions de coût du même ordre sans compromis de qualité revendiqué : facteur deux pour ROUTELLM [28], jusqu'à 98 % pour FRUGALGPT [10], 40 % pour HYBRIDLLM [32], plus de 50 % pour AUTOMIX [33]. Notre réduction de 30 % se situe dans cet ordre de grandeur, proche de HYBRIDLLM.

La différence tient à un choix de reporting : nous assumons un écart de qualité mesuré (0,88 contre 0,78) plutôt qu'un "sans compromis" obtenu ailleurs en fixant un plancher implicite bas. Deux raisons resserrent par ailleurs notre marge : un pool volontairement réduit à deux modèles, là où FRUGALGPT cascade plusieurs paliers, et une métrique sévère sur le complexe (§5.3.2).

La confrontation reste indicative. Aucun de ces routers n'est évalué sur un domaine réseau ni sous contrainte dure de latence par requête, et aucun ne rapporte la sensibilité de son gain à la composition du flux traité (§5.6.1), alors que nos mesures la montrent déterminante.

5.7 Spécialisation par domaine

La troisième contribution suppose qu'un modèle cadré sur un domaine réseau répond mieux, sur ce domaine, qu'un modèle générique de même capacité. Le prototype initial en tenait pour acquis le bénéfice sans l'avoir construit : un "spécialiste" y désignait le modèle lourd sous un identifiant suffixé, et le cadrage envoyé restait celui du générique. Nous mesurons ici l'effet réel d'un cadrage d'expertise. Le protocole oppose deux bras sur les mêmes intents, notés par le même juge contre la même checklist : seul le cadrage change. Sur son domaine, le spécialiste porte la qualité de 0,924 à 0,962 sur quinze intents RAN. Hors de son domaine, le même cadrage appliqué à des intents de cœur de réseau la fait tomber à 0,777, contre 0,914 pour le générique.

Table 5.10: Effet d'un cadrage d'expertise, écart sur le générique.

Cadrage	Générique	Spécialiste	Écart
Sur son domaine (15 intents RAN)	0,924	0,962	+0,038
Hors domaine (6 intents cœur)	0,914	0,777	−0,137

L'asymétrie est le résultat, davantage que le gain. La pénalité hors domaine vaut 3,6 fois le bénéfice sur le domaine, ce qui change le statut de cette contribution : la spécialisation n'est pas un supplément que l'on ajoute à un pool en espérant mieux, c'est une option dont le rendement dépend entièrement de la fiabilité du routage par domaine. Un système qui se trompe de spécialiste perd davantage qu'un système qui n'en emploie aucun. La contribution C2 est ainsi subordonnée à C1, et non complémentaire comme le laissait entendre la formulation initiale.

Ces valeurs remplacent, dans la politique de routage, deux constantes posées a priori. Elles y figurent comme écarts sur le générique plutôt que comme niveaux absolus : la comparaison appariée rend l'écart fiable, là où le niveau dépend du jeu d'intents retenu. Le coût d'un spécialiste étant celui de son modèle de base, le routeur ne le retient qu'à coût égal, la qualité servant alors de critère de départage.

5.8 Validation en environnement réel

Au-delà du juge, qui évalue si une réponse *paraît* correcte, nous mesurons un **taux de réalisation d'intent** : la fraction des intents qui, une fois le plan appliqué sur un réseau, produisent le comportement voulu, vérifié dans le plan de données. Le banc émule un réseau avec Mininet [50] pilotant des commutateurs Open vSwitch [51], sans contrôleur externe : le modèle cible produit lui-même les opérations, que le banc traduit en règles OpenFlow.

5.8.1 Le banc

Le modèle cible produit un plan de plusieurs opérations sur sept verbes : autoriser et bloquer, y compris sur un port applicatif précis, plafonner un débit, dupliquer un flux vers une sonde, imposer un chemin, marquer une classe de priorité.

La topologie devient un losange à quatre commutateurs offrant deux chemins, ce qui rend le reroutage observable, avec un hôte sonde dédié à la duplication. Les commutateurs fonctionnent en mode **secure** et le banc installe lui-même son acheminement de base : le plan de données ne dépend plus d'un

5.8. VALIDATION EN ENVIRONNEMENT RÉEL

apprentissage d’adresses, donc il est déterministe. Le jeu compte 24 intents répartis à parts égales entre simples, moyens et complexes, portant 44 vérifications de huit natures.

5.8.2 Validation du banc lui-même

Un banc de mesure peut produire des chiffres plausibles sans dépendre de ce que le modèle a fait. Nous soumettons donc chaque vérification à deux contrôles symétriques, exécutés sur l’ensemble du jeu.

Le contrôle négatif rejoue les 24 intents avec un plan syntaxiquement valide mais sans effet réseau. Toute vérification qui passe alors ne mesure pas le modèle. Le contrôle positif rejoue les mêmes intents avec le plan dérivé de leur vérité terrain. Toute vérification qui échoue alors ne peut être satisfaite par aucun modèle. Les deux sont gratuits, puisque ni l’un ni l’autre n’appelle de modèle.

Table 5.11: Double contrôle du banc, taux d’intents entièrement réalisés.

Contrôle	Simple	Moyen	Complexe	Global
Négatif (plan sans effet)	0 %	0 %	0 %	0 %
Positif (plan de référence)	100 %	100 %	100 %	100 %

La première exécution de ce dispositif a invalidé plusieurs vérifications que la seule relecture du code avait laissées passer. Un test de chemin interrogeait le chemin par défaut, donc il était satisfait sans action du modèle. Un test de duplication comptait le trafic de signalisation de la pile réseau plutôt que les paquets copiés. Le cache de la voie rapide d’Open vSwitch servait enfin le comportement antérieur au plan aux mesures lancées immédiatement après son application.

Aucun de ces défauts ne provoquait d’erreur. Chacun produisait un résultat lisible et faux, orienté dans le sens attendu par l’hypothèse de travail : c’est précisément le régime où une validation conçue par l’auteur de la méthode se vérifie elle-même.

5.8.3 Résultats

La campagne oppose les trois stratégies sur le pool de production, le léger `qwen-2.5-72b` contre le lourd `claude-opus-4.8`. Le routeur délègue 7 intents sur 24 au léger et 17 au lourd, si bien que la politique de routage s’exerce réellement sur ce jeu plutôt que de se réduire à l’un de ses deux tiers.

Table 5.12: Taux d’intents entièrement réalisés dans le plan de données, par strate de complexité annotée.

Stratégie	Simple	Moyen	Complexe	Global
Always-Heavy	100 %	75 %	62 %	79 %
InferRouter-LLM	88 %	75 %	62 %	75 %
Always-Light	75 %	62 %	62 %	67 %

Le classement global reproduit celui du benchmark par juge, cette fois mesuré dans le plan de données : le routage se place près du modèle fort tout en déléguant près d’un tiers des intents au modèle

5.9. PROBLÈMES RENCONTRÉS

économique. La décroissance du taux avec la complexité est un résultat et non un artefact : le contrôle positif réalise 100 % sur les trois strates, donc aucun de ces échecs ne tient au banc.

Ces quatre points de réalisation cédés au tout-lourd achètent une économie. Aux tarifs unitaires du pool (tableau 5.3), la répartition mesurée de 7 intents vers le léger et 17 vers le lourd représente 29 % de coût en moins que le tout-lourd sur ce jeu. L'ordre de grandeur rejoint celui du benchmark par juge, ce qui n'allait pas de soi : la répartition découle ici de la complexité prédite sur des intents réellement hétérogènes, non d'un jeu dominé par les cas simples.

Le léger produit aussi un plan inexploitable sur un intent des 24, là où le lourd n'en produit aucun. L'échec apparaît à l'analyse syntaxique, avant toute application réseau, et se comptabilise comme une non-réalisation.

Chaque stratégie appelant son propre modèle, la variabilité d'un modèle entre deux appels se mêle à l'effet du routage. Nous isolons donc la décision : le routeur choisit un tiers, et la mesure réutilise le plan déjà produit par ce tiers. Sans cette précaution, InferRouter-LLM tombait sous ses deux composants sur la strate complexe (50 % contre 62 %), un artefact de tirage que la reprise par sélection fait disparaître à taux global inchangé.

Deux réserves bornent la lecture. Chaque cellule compte huit intents, si bien qu'un intent pèse 12,5 points : seuls les taux globaux se comparent, pas les écarts à l'intérieur d'une strate. Le léger égale ensuite le lourd sur le complexe alors qu'il perd 25 points sur le simple, ce que cet effectif ne permet pas d'interpréter.

5.8.4 Portée et limites de la mesure

Le banc ne mesure pas tout ce que le vocabulaire exprime. La garantie de débit minimal reste hors périmètre : sa réalisation demande de placer une file d'attente sur le lien d'étranglement, ce qui exigerait que les modules de traduction connaissent la topologie, au prix de l'indépendance qui fait leur simplicité. Le verbe correspondant demeure accepté en entrée, sans vérification associée. La priorité est contrôlée sur le marquage des paquets et non sur l'ordonnancement sous charge, ce dernier recouvrant la garantie de débit. Une vérification de joignabilité, enfin, ne discrimine qu'accompagnée d'une interdiction : seule, elle constate une connectivité que la topologie fournit déjà.

Le périmètre reste celui des primitives exprimables en OpenFlow sur une topologie à quatre commutateurs. Les intents de corrélation multi-domaines continuent de relever de l'évaluation par juge.

5.9 Problèmes rencontrés

La validation en environnement réel a demandé trois tentatives avant d'aboutir à un banc dont les mesures soient interprétables. Les impasses sont instructives, et leur coût a dépassé celui de la solution retenue.

5.9.0.0.1 Les cadres logiciels envisagés se sont révélés inexploitables. Le pipeline NetIntent, qui promettait un pont déjà fait entre LLM et actions de contrôleur, n'a aucun code publié : l'article seul ne suffit pas à rejouer une expérience. Le contrôleur SDN ONOS ne démarre pas sur l'architecture ARM du poste de travail, deux bibliothèques natives n'existant pas pour cette cible, et l'émulation x86 fait échouer son cache de modules. Le contrôleur Ryu, enfin, ne s'installe plus sur les versions récentes

de Python. Nous avons donc abandonné l'idée d'un contrôleur externe et appliqué les opérations directement sur Open vSwitch, InferRouter-LLM tenant lui-même le rôle de décideur.

5.9.0.0.2 Un premier banc mesurait une question mal posée. Sa sortie était contrainte à une action unique entre deux points. Cette forme, adoptée pour aller vite, imposait à tout intent testable de ne toucher qu'une entité sous une seule contrainte, donc d'être simple par construction. Le routeur y classait l'intégralité du jeu en simple et reproduisait exactement le tier léger ; la branche qui délègue au modèle fort, c'est-à-dire la moitié de la politique évaluée, n'était jamais exercée. Les taux obtenus alors ne sont pas repris ici : ils décrivent un dispositif qui ne pouvait pas répondre à la question posée.

5.9.0.0.3 Le banc étendu produisait des chiffres crédibles et faux. Cinq défauts de mesure ont été identifiés après sa mise au point, dont trois par le double contrôle décrit au §5.8 et deux par relecture. Aucun ne provoquait d'erreur visible. Un analyseur syntaxique trop gourmand rejetait des plans corrects dès que la réponse comportait une phrase d'introduction, ce qui pénalise le modèle le plus verbeux, donc le léger. Un plan comportant une opération malformée était silencieusement tronqué à sa première opération valide puis compté en succès partiel, ce qui pénalise les plans longs, donc les intents complexes.

Ces deux biais orientaient l'erreur dans le sens de l'hypothèse de travail. Nous en tirons la règle appliquée au §5.8 : un instrument conçu par l'auteur de la méthode qu'il évalue doit lui-même être validé, et les contrôles négatif et positif fournissent cette validation à coût nul.

5.10 Synthèse et limites

Trois acquis ressortent de cette campagne. La complexité d'un intent porte un signal sémantique réel une fois la longueur neutralisée : 65 à 73 % sur les attributs calculés, jusqu'à 85 % avec les embeddings de phrase, pour une baseline de 33 %. Un juge local **gemma2:9b** avec RocketEval suffit pour le routage, qui ne demande qu'une discrimination grossière, validée à 100 %. Enfin, le routage produit un compromis coût-qualité réel : InferRouter-LLM réduit le coût de 30 % face à Always-Heavy pour une baisse de qualité bornée ($0,88 \rightarrow 0,78$), et surpasse nettement Random en qualité (0,78 contre 0,65). Les contributions C1 et C3 tiennent pour leur usage visé, et C2, reformulée en minimisation de coût sous plancher de qualité, produit un arbitrage mesuré dont l'opérateur règle le point de fonctionnement le long d'une frontière de Pareto.

Un enseignement transversal se dégage : la valeur du routage est bornée des deux côtés par le pool. Un léger trop faible l'annule (tout part au lourd, §5.4.1) ; un lourd trop faible aussi (le léger l'égale, plus rien à arbitrer, §5.4.4). Un léger trop *uniforme*, enfin, l'annule d'une troisième manière (§5.6) : si sa faiblesse ne suit aucun signal estimable, le routage informé ne bat plus le hasard, même avec un léger supérieur. Le routage crée de la valeur dans une zone étroite : un différentiel de qualité doit exister entre les tiers, et surtout varier avec un attribut de l'intent que l'estimateur sait prédire.

Un point reste non établi : le juge local ne sert pas d'oracle de qualité fine. Sur vingt paires opposant une réponse correcte à une variante dégradée par une valeur inventée, une étape manquante ou une affirmation fausse, il rend vingt égalités. Ni la taille du modèle local ni la notation par paires ne le corrigent.

La capacité du modèle, elle, y contribue. Soumis au même protocole apparié, un juge fort accessible par API désigne la bonne réponse dans dix cas sur vingt. La discrimination fine n'est donc pas hors

de portée par nature, elle est hors de portée d'un juge local ; elle reste néanmoins partielle, puisque la moitié des erreurs injectées échappe encore à ce juge. Cette limite borne la lecture absolue de l'AIQ sur le benchmark, en particulier sur le complexe, et non le classement des stratégies, qui ne demande qu'une discrimination grossière.

Le travail futur découle de cette limite et du périmètre retenu. La mesure fine de qualité reste ouverte : un juge fort double la discrimination sans la résoudre, si bien qu'un oracle exigerait vraisemblablement autre chose qu'un juge plus gros, par exemple une vérification ancrée sur l'état du réseau plutôt que sur le texte, dans la ligne du banc du §5.8.

Deux extensions du pool se dessinent. Une cascade à plusieurs paliers rapprocherait vraisemblablement InferRouter-LLM des chiffres de FRUGALGPT. Un palier rapide, ensuite, ouvrirait un axe que le pool actuel ne peut pas servir : ses deux tiers répondent en 5,7 et 10,0 secondes, quand un modèle économique récent répond en 1,3 seconde. Ce palier ne franchirait toutefois que le plancher le plus bas, sa qualité sur les intents simples valant 0,42 contre 0,64 pour le tier léger retenu : le gain de latence serait donc confiné aux intents simples et peu critiques.

Un jeu d'évaluation plus large libérerait enfin les mesures de la contrainte de budget qui borne cette campagne à quelques dizaines d'intents selon les expériences.

Chapter 6

Conclusion

6.1 Synthèse

Ce mémoire a posé une question opérationnelle : comment traiter un flux d'intents réseau exprimés en langage naturel avec des LLM, sans payer le prix d'un modèle frontière sur chaque requête. La réponse proposée, InferRouter-LLM, route chaque intent vers le modèle le moins coûteux qui garantit une qualité minimale acceptable, en s'appuyant sur trois briques : un estimateur de complexité sémantique, un routeur sous contraintes, et un juge LLM local qui mesure la qualité sans intervention humaine.

Le système couvre la chaîne de bout en bout, évalué sur 252 intents télécom générés puis revus. Chaque chiffre provient d'un run daté et rejouable (méthodologie code-first). Cette discipline a plusieurs fois évité des résultats trompeurs : l'exactitude de 99,6 % de l'estimateur était un artefact de longueur, le gain de coût du routage a dû être requalifié en compromis assumé plutôt qu'en "même qualité pour moins cher", et le banc de validation réelle a livré plusieurs vérifications qui produisaient un résultat lisible sans dépendre du modèle évalué. Ce dernier point a motivé le double contrôle décrit au §5.8 : un instrument conçu par l'auteur de la méthode qu'il évalue doit être validé avant d'être cru.

6.2 Apports scientifiques

6.2.0.0.1 Un estimateur de complexité ancré sur le fond. La complexité d'un intent porte un signal sémantique réel, indépendant de sa longueur : 65 à 73 % d'exactitude sur les attributs calculés (entités, contraintes, domaines croisés), jusqu'à 85 % avec les embeddings de phrase, contre une baseline de 33 %. La neutralisation du biais de longueur, obtenue en régénérant le jeu à longueur contrôlée, distingue ce résultat des mesures naïves.

6.2.0.0.2 Un routeur reformulé en minimisation de coût sous plancher. La formulation initiale, "choisir la plus haute qualité attendue", s'est avérée inadaptée face à un lourd nettement supérieur : elle dégénère en stratégie always-heavy. Le renversement primal-dual retenu, minimiser le coût sous un plancher de qualité indexé sur la criticité, produit un compromis mesuré : face à always-heavy, une réduction de coût réel de 30 % pour une baisse de qualité bornée (0,88 à 0,78), et une supériorité nette sur un routage aléatoire. Le plancher devient le paramètre par lequel l'opérateur choisit son point de fonctionnement sur une frontière de Pareto.

6.3. LIMITES ET PERSPECTIVES

Ce taux d'économie n'est toutefois pas une constante du système. Rejoué sur un autre échantillon, il tombe à 11 % à qualité inchangée, parce que seuls les intents à la fois simples et peu critiques descendent au tier économique (§5.6.1). L'économie mesure donc autant la composition du flux d'intents que la politique elle-même, ce qui est le comportement attendu d'un arbitrage indexé sur la criticité.

6.2.0.0.3 Un juge local fiable pour le routage, borné comme oracle. Un juge `gemma2:9b` muni de la méthode `RocketEval` atteint 100 % en discrimination grossière, ce dont le routage a besoin, mais ne détecte pas une erreur isolée dans une réponse sinon correcte : sur vingt paires opposant une réponse juste à une variante dégradée, il rend vingt égalités. Ni un juge local plus gros ni la notation par paires ne corrigent ce plafond. Un juge fort accessible par API, soumis au même protocole, tranche dans la moitié des cas : la capacité du modèle déplace donc la limite sans la lever. Le juge local sert de signal de routage, non d'oracle de qualité fine.

6.2.0.0.4 La spécialisation ne paie que si le routage vise juste. Un cadrage d'expertise par domaine améliore la qualité sur son domaine, de 0,924 à 0,962, et la dégrade nettement hors de lui, à 0,777. La pénalité vaut 3,6 fois le gain. Cette asymétrie change le statut de la spécialisation : elle n'est pas un supplément que l'on ajoute à un pool en espérant mieux, mais une option dont le rendement dépend entièrement de la fiabilité du routage par domaine. Un système qui se trompe de spécialiste perd davantage qu'un système qui n'en utilise aucun.

6.2.0.0.5 Le bon léger n'est pas le plus fort. Le résultat le plus contre-intuitif du travail concerne le choix du modèle économique. Une étude comparée de six modèles a montré que le meilleur léger dans l'absolu (`deepseek-v3.2`, plus juste et moins cher) *annule* la valeur du routage : son profil uniforme ne laisse aucun signal indiquant où le lourd apporte le plus, et `InferRouter-LLM` y fait moins bien qu'un tirage aléatoire. Un léger dont la faiblesse suit la complexité, même globalement inférieur, rétablit ce signal et fait battre le hasard au routage. Le critère de sélection d'un léger n'est donc pas sa qualité absolue mais la corrélation de sa faiblesse avec un attribut estimable de l'intent. Ce constat nuance une hypothèse implicite de la littérature du routage, qui traite le modèle économique comme interchangeable pourvu qu'il soit assez bon.

6.3 Limites et perspectives

Plusieurs limites circonscrivent la portée de ces résultats. Le juge ne mesure pas la qualité fine, ce qui borne la lecture absolue des scores, surtout sur les intents complexes ; un juge fort double la discrimination sans la résoudre, si bien qu'une évaluation précise demanderait vraisemblablement un autre ancrage que le texte. Aucun modèle intermédiaire ne s'est inséré entre le léger et le lourd, le juge saturant sur toute la gamme de milieu. Enfin, les mesures reposent sur quelques dizaines d'intents selon les expériences, contrainte imposée par le budget d'appels, sans intervalle de confiance ; les écarts rapportés doivent se lire comme des ordres de grandeur, non comme des mesures fines.

Le périmètre lui-même est assumé : `InferRouter-LLM` est une couche de sélection de modèle en amont de la phase de traduction d'une boucle IBN, et ne pousse aucune configuration sur l'infrastructure. L'actuation exigerait la validation formelle des phases d'activation et d'assurance, hors champ de ce travail.

6.3. LIMITES ET PERSPECTIVES

Trois prolongements se dégagent. La mesure de qualité fine reste ouverte : puisque grossir le juge ne suffit pas, une piste consiste à l'ancrer sur l'effet réel d'une réponse plutôt que sur son texte, dans la ligne du banc du §5.8. Un pool enrichi, avec un palier intermédiaire et un palier rapide, lisserait la frontière de Pareto et rapprocherait le système des cascades multi-modèles de l'état de l'art ; le palier rapide ouvrirait de surcroît un axe de latence que le pool actuel ne peut pas servir. Enfin, la validation en environnement réel (§5.8) gagnerait à être étendue à un contrôleur SDN de production et à des primitives dépassant celles qu'une topologie à quatre commutateurs permet d'exprimer.

Bibliography

- [1] N. Maslej, L. Fattorini, R. Perrault, Y. Gil, V. Parli, N. Kariuki, E. Capstick, A. Reuel, E. Brynjolfsson, J. Etchemendy, K. Ligett, T. Lyons, J. Manyika, J. C. Niebles, Y. Shoham, R. Wald, T. Walsh, A. Hamrah, L. Santarlasci, J. B. Lotufo, A. Rome, A. Shi, and S. Oak, “Artificial Intelligence Index Report 2025,” AI Index Steering Committee, Institute for Human-Centered AI, Stanford Univ., Stanford, CA, USA, Tech. Rep., Apr. 2025, doi: 10.48550/arXiv.2504.07139. [Online]. Available: <https://hai.stanford.edu/ai-index/2025-ai-index-report>
- [2] A. Maatouk, N. Piovesan, F. Ayed, A. De Domenico, and M. Debbah, “Large language models for telecom: Forthcoming impact on the industry,” *IEEE Commun. Mag.*, vol. 63, no. 1, pp. 62–68, Jan. 2024, doi: 10.1109/MCOM.001.2300473.
- [3] M. El Rajab, L. Yang, and A. Shami, “Zero-touch networks: Towards next-generation network automation,” *Comput. Netw.*, vol. 243, p. 110294, Apr. 2024, doi: 10.1016/j.comnet.2024.110294.
- [4] E. Karakaya, O. Ercetin, H. Ozkan, M. Karaca, E. Dehghan Biyar, and A. Palaios, “Online learning for autonomous management of intent-based 6G networks,” arXiv preprint arXiv:2407.17767, Jul. 2024, doi: 10.48550/arXiv.2407.17767.
- [5] A. Clemm, L. Ciavaglia, L. Z. Granville, and J. Tantsura, “Intent-based networking - concepts and definitions,” Internet Research Task Force (IRTF), RFC 9315, Oct. 2022, doi: 10.17487/RFC9315. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9315>
- [6] M. Falkner and J. Apostolopoulos, “Intent-based networking for the enterprise: A modern network architecture,” *Commun. ACM*, vol. 65, no. 11, pp. 108–117, Nov. 2022, doi: 10.1145/3538513.
- [7] A. Leivadeas and M. Falkner, “A survey on intent-based networking,” *IEEE Commun. Surveys Tuts.*, vol. 25, no. 1, pp. 625–655, First Quarter 2023, doi: 10.1109/COMST.2022.3215919.
- [8] K. Mehmood, K. Kralevska, and D. Palma, “Intent-driven autonomous network and service management in future cellular networks: A structured literature review,” *Comput. Netw.*, vol. 220, p. 109477, Jan. 2023, doi: 10.1016/j.comnet.2022.109477.
- [9] Y. Huang, H. Du, X. Zhang, D. Niyato, J. Kang, Z. Xiong, S. Wang, and T. Huang, “Large language models for networking: Applications, enabling techniques, and challenges,” *IEEE Netw.*, vol. 38, no. 5, pp. 235–242, Sep. 2024, doi: 10.1109/MNET.2024.3435752.
- [10] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” *Trans. Mach. Learn. Res. (TMLR)*, May 2024, doi: 10.48550/arXiv.2305.05176.
- [11] J. W.-K. Hong, “A comprehensive survey on LLM-based network management and operations,” *Int. J. Netw. Manag.*, vol. 35, no. 6, p. e70029, Nov. 2025, doi: 10.1002/nem.70029.
- [12] T. Wei, W. Wen, R. Qiao, X. Sun, and J. Ma, “RocketEval: Efficient automated LLM evaluation via grading checklist,” in *Proc. 13th Int. Conf. Learn. Representations (ICLR)*, Singapore, Apr. 2025, doi: 10.48550/arXiv.2503.05142.
- [13] L. Pang, C. Yang, D. Chen, Y. Song, and M. Guizani, “A survey on intent-driven networks,” *IEEE Access*, vol. 8, pp. 22 862–22 873, Jan. 2020, doi: 10.1109/ACCESS.2020.2969208.
- [14] *Zero-touch network and Service Management (ZSM); Intent-driven autonomous networks; Generic aspects*, ETSI GR ZSM 011, Rev. 1.1.1, Feb. 2023. [Online]. Available: https://www.etsi.org/deliver/etsi_gr/ZSM/001_099/011/01.01.01_60/gr_ZSM011v010101p.pdf
- [15] E. Zeydan and Y. Turk, “Recent advances in intent-based networking: A survey,” *Proc. IEEE 91st Veh. Technol. Conf. (VTC2020-Spring)*, pp. 1–5, May 2020, doi: 10.1109/VTC2020-Spring48590.2020.9128422.

BIBLIOGRAPHY

- [16] A. S. Jacobs, R. J. Pfitscher, R. H. Ribeiro, R. A. Ferreira, L. Z. Granville, W. Willinger, and S. G. Rao, “Hey, Lumi! Using natural language for intent-based network management,” in *Proc. 2021 USENIX Annu. Tech. Conf. (USENIX ATC '21)*. USENIX Association, Jul. 2021, pp. 625–639. [Online]. Available: <https://www.usenix.org/conference/atc21/presentation/jacobs>
- [17] C. Liu, X. Xie, X. Zhang, and Y. Cui, “Large language models for networking: Workflow, advances and challenges,” arXiv preprint arXiv:2404.12901, Apr. 2024, doi: 10.48550/arXiv.2404.12901.
- [18] H. Zhou, C. Hu, Y. Yuan, Y. Cui, Y. Jin, C. Chen, H. Wu, D. Yuan, L. Jiang, D. Wu, X. Liu, C. Zhang, X. Wang, and J. Liu, “Large language model (LLM) for telecommunications: A comprehensive survey on principles, key techniques, and opportunities,” arXiv preprint arXiv:2405.10825, May 2024, doi: 10.48550/arXiv.2405.10825.
- [19] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, “NetLLM: Adapting large language models for networking,” in *Proc. ACM SIGCOMM 2024 Conf.*, Sydney, Australia, Aug. 2024, pp. 661–678, doi: 10.1145/3651890.3672268.
- [20] D. M. Manias, A. Chouman, and A. Shami, “Semantic routing for enhanced performance of LLM-assisted intent-based 5G core network management and orchestration,” in *Proc. IEEE Global Commun. Conf. (GLOBECOM)*, Cape Town, South Africa, Dec. 2024, pp. 1–6, doi: 10.1109/GLOBECOM52923.2024.10901594.
- [21] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, “NetConfEval: Can LLMs facilitate network configuration?” *Proc. ACM Netw.*, vol. 2, no. CoNEXT2, p. Art. no. 7, Jun. 2024, doi: 10.1145/3656296.
- [22] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, “What do LLMs need to synthesize correct router configurations?” in *Proc. 22nd ACM Workshop Hot Topics Networks (HotNets '23)*, Cambridge, MA, USA, Nov. 2023, pp. 189–195, doi: 10.1145/3626111.3628194.
- [23] B. Ifland, E. Duani, R. Krief, M. Ohana, A. Zilberman, A. Murillo, O. Manor, O. Lavi, H. Kenji, A. Shabtai, Y. Elovici, and R. Puzis, “GeNet: A multimodal LLM-based co-pilot for network topology and configuration,” arXiv preprint arXiv:2407.08249, Jul. 2024, doi: 10.48550/arXiv.2407.08249.
- [24] F. Ayed, A. Maatouk, N. Piovesan, A. De Domenico, M. Debbah, and Z.-Q. Luo, “Hermes: A large language model framework on the journey to autonomous networks,” arXiv preprint arXiv:2411.06490, Nov. 2024, doi: 10.48550/arXiv.2411.06490.
- [25] D. M. Manias, A. Chouman, and A. Shami, “Towards intent-based network management: Large language models for intent extraction in 5G core networks,” in *Proc. 2024 20th Int. Conf. Design Reliable Commun. Netw. (DRCN)*, Montreal, Canada, May 2024, pp. 1–6, doi: 10.1109/DRCN60692.2024.10539183.
- [26] A. Mekrache and A. Ksentini, “LLM-enabled intent-driven service configuration for next generation networks,” in *Proc. 2024 IEEE 10th Int. Conf. Netw. Softwarization (NetSoft)*, Saint Louis, MO, USA, Jun. 2024, pp. 253–257, doi: 10.1109/NetSoft60951.2024.10588881.
- [27] Y. Wei, X. Xie, T. Hu, Y. Zuo, X. Chen, K. Chi, and Y. Cui, “INTA: Intent-based translation for network configuration with LLM agents,” in *Proc. IEEE 33rd Int. Conf. Netw. Protocols (ICNP)*, Oct. 2025, doi: 10.48550/arXiv.2501.08760.
- [28] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, “RouteLLM: Learning to route LLMs from preference data,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, Singapore, Apr. 2025, doi: 10.48550/arXiv.2406.18665.
- [29] K. Lu, H. Yuan, R. Lin, J. Lin, Z. Yuan, C. Zhou, and J. Zhou, “Routing to the expert: Efficient reward-guided ensemble of large language models,” in *Proc. 2024 Conf. North Amer. Chapter Assoc. Comput. Linguistics: Human Lang. Technol. (NAACL-HLT)*, Mexico City, Mexico, Jun. 2024, pp. 1964–1974, doi: 10.18653/v1/2024.naacl-long.109.
- [30] S. Chen, W. Jiang, B. Lin, J. T. Kwok, and Y. Zhang, “RouterDC: Query-based router by dual contrastive learning for assembling large language models,” in *Proc. 38th Conf. Neural Inf. Process. Syst. (NeurIPS)*, Vancouver, Canada, Dec. 2024, doi: 10.48550/arXiv.2409.19886.
- [31] S. N. Hari and M. Thomson, “Tryage: Real-time, intelligent routing of user prompts to large language models,” arXiv preprint arXiv:2308.11601, Aug. 2023, doi: 10.48550/arXiv.2308.11601.
- [32] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Ruhle, L. V. S. Lakshmanan, and A. H. Awadallah, “Hybrid LLM: Cost-efficient and quality-aware query routing,” in *Proc. Int. Conf. Learning Representations (ICLR)*, Vienna, Austria, May 2024, doi: 10.48550/arXiv.2404.14618.

BIBLIOGRAPHY

- [33] A. Madaan, P. Aggarwal, A. Anand, S. P. Potharaju, S. Mishra, P. Zhou, A. Gupta, D. Rajagopal, K. Kappaganthu, Y. Yang, S. Upadhyay, Mausam, and M. Faruqui, “AutoMix: Automatically mixing language models,” in *Proc. 38th Conf. Neural Inf. Process. Syst. (NeurIPS)*, Vancouver, Canada, Dec. 2024, doi: 10.48550/arXiv.2310.12963.
- [34] D. Jiang, X. Ren, and B. Y. Lin, “LLM-Blender: Ensembling large language models with pairwise ranking and generative fusion,” in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (ACL), Vol. 1 (Long Papers)*, Toronto, Canada, Jul. 2023, pp. 14 165–14 178, doi: 10.18653/v1/2023.acl-long.792.
- [35] Z. Zhao, S. Jin, and Z. M. Mao, “Eagle: Efficient training-free router for multi-LLM inference,” in *Proc. 38th Conf. Neural Inf. Process. Syst. (NeurIPS) Workshops*, Vancouver, Canada, Dec. 2024, doi: 10.48550/arXiv.2409.15518.
- [36] T. Shnitzer, A. Ou, M. Silva, K. Soule, Y. Sun, J. Solomon, N. Thompson, and M. Yurochkin, “Large language model routing with benchmark datasets,” in *Proc. Conf. Lang. Model. (COLM)*, Philadelphia, PA, USA, Oct. 2024, doi: 10.48550/arXiv.2309.15789.
- [37] J. Dekoninck, M. Baader, and M. Vechev, “A unified approach to routing and cascading for LLMs,” arXiv preprint arXiv:2410.10347, Oct. 2024, doi: 10.48550/arXiv.2410.10347.
- [38] J. Zhang, R. Krishna, A. H. Awadallah, and C. Wang, “EcoAssistant: Using LLM assistant more affordably and accurately,” in *Proc. COLM Workshop / arXiv*, Oct. 2023, doi: 10.48550/arXiv.2310.03046.
- [39] W. Song, Z. Huang, C. Cheng, W. Gao, B. Xu, G. Zhao, F. Wang, and R. Wu, “IRT-Router: Effective and interpretable multi-LLM routing via Item Response Theory,” in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (ACL)*, Vienna, Austria, 2025, doi: 10.48550/arXiv.2506.01048.
- [40] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, S. Wang, K. Zhang, Y. Wang, W. Gao, L. Ni, and J. Guo, “A survey on LLM-as-a-judge,” arXiv preprint arXiv:2411.15594, Nov. 2024, doi: 10.48550/arXiv.2411.15594.
- [41] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, “BERTScore: Evaluating text generation with BERT,” in *Proc. 8th Int. Conf. Learn. Representations (ICLR)*, Addis Ababa, Ethiopia, Apr. 2020, doi: 10.48550/arXiv.1904.09675. [Online]. Available: <https://openreview.net/forum?id=SkeHuCVFDr>
- [42] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, “G-Eval: NLG evaluation using GPT-4 with better human alignment,” in *Proc. 2023 Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, Singapore, Dec. 2023, pp. 2511–2522, doi: 10.18653/v1/2023.emnlp-main.153.
- [43] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-Bench and Chatbot Arena,” in *Proc. 37th Conf. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track*, New Orleans, LA, USA, Dec. 2023, doi: 10.48550/arXiv.2306.05685.
- [44] L. Zhu, X. Wang, and X. Wang, “JudgeLM: Fine-tuned large language models are scalable judges,” in *Proc. 13th Int. Conf. Learn. Representations (ICLR), Spotlight*, Singapore, Apr. 2025, doi: 10.48550/arXiv.2310.17631.
- [45] Y. Wang, Z. Yu, Z. Zeng, L. Yang, C. Wang, H. Chen, C. Jiang, R. Xie, J. Wang, X. Xie, W. Ye, S. Zhang, and Y. Zhang, “PandaLM: An automatic evaluation benchmark for LLM instruction tuning optimization,” in *Proc. 12th Int. Conf. Learn. Representations (ICLR)*, Vienna, Austria, May 2024, doi: 10.48550/arXiv.2306.05087.
- [46] 3GPP, “Service requirements for the 5G system; Stage 1,” 3rd Generation Partnership Project (3GPP), Tech. Spec. (TS) 22.261, Sep. 2023, rel. 18, V18.6.0. [Online]. Available: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261
- [47] Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, and S. K. Upadhyay, “RouterBench: A benchmark for multi-LLM routing system,” in *Proc. Int. Conf. Mach. Learn. (ICML) Workshops*, Vienna, Austria, Jul. 2024, doi: 10.48550/arXiv.2403.12031.
- [48] Jiang *et al.*, “Agentic AI Empowered Intent-Based Networking for 6G,” arXiv preprint arXiv:2601.06640, 2025, doi: 10.48550/arXiv.2601.06640.
- [49] B. Saha, “IBNs: Intent-Based Networking dataset generator,” <https://github.com/barun-saha/IBNs>, 2024, open-source pipeline for generating natural-language network intents.
- [50] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: rapid prototyping for software-defined networks,” in *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets)*. ACM, 2010, pp. 1–6, doi: 10.1145/1868447.1868466.
- [51] B. Pfaff, J. Pettit, T. Koponen, E. J. Jackson, A. Zhou, J. Rajahalme, J. Gross, A. Wang, J. Stringer, P. Shelar, K. Amidon, and M. Casado, “The design and implementation of Open vSwitch,” in *Proceedings of the 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. USENIX Association, 2015, pp. 117–130.

Acronyms

3GPP	Third Generation Partnership Project.	5
ACL	Access Control List.	5
AIQ	Average Inference Quality.	39
AMF	Access and Mobility Management Function.	6
API	Application Programming Interface.	3, 22, 35, 36, 38, 43, 46
BLEU	Bilingual Evaluation Understudy.	11, 55
DSL	Domain-Specific Language.	6
eMBB	enhanced Mobile BroadBand.	5
ETSI	European Telecommunications Standards Institute.	5
GPU	Graphics Processing Unit.	17, 22
IBN	Intent-Based Networking.	1, 4, 17, 23, 24
IBNS	Intent-Based Networking System.	4
IBSec	Intent-Based Security.	5
IDS	Intrusion Detection System.	5
IRT	Item Response Theory.	19
IRTF	Internet Research Task Force.	1, 4, 5
JSON	JavaScript Object Notation.	8
KPI	Key Performance Indicator.	18, 30
LLM	Large Language Model.	1, 7, 42
LoRA	Low-Rank Adaptation.	7
MCS	Modulation and Coding Scheme.	5
mMTC	massive Machine Type Communications.	5
NER	Named Entity Recognition.	6, 18
NF	Network Function.	6, 18
NLP	Natural Language Processing.	6
NSD	Network Service Descriptor.	8
NSI	Network Slice Instance.	5

ACRONYMS

- OWL** Web Ontology Language. 6
- PCF** Policy Control Function. 6
- POMDP** Partially Observable Markov Decision Process. 10
- QoS** Quality of Service. 6
- RAG** Retrieval-Augmented Generation. 7
- RAN** Radio Access Network. 5, 40
- RIC** RAN Intelligent Controller. 5
- ROUGE** Recall-Oriented Understudy for Gisting Evaluation. 11, 55
- SDN** Software Defined Networking. 29, 42, 47
- SLA** Service Level Agreement. 5
- SMF** Session Management Function. 6
- SW-ranking** Similarity-Weighted ranking. 9
- UE** User Equipment. 33
- UPF** User Plane Function. 6
- uRLLC** Ultra-Reliable Low-Latency Communications. 5, 14, 18
- ZSM** Zero-touch network and Service Management. 5

Appendix A

Annexes techniques

A.1 Configuration du système

Toutes les mesures rapportées dans ce mémoire sont reproductibles à partir de la configuration ci-dessous. Les artefacts (jeux d'intents, résultats de calibration, modèle d'estimateur persisté) sont versionnés dans le dépôt du projet.

A.1.0.0.1 Matériel. MacBook Air M5 (24 Go de mémoire unifiée, SSD 1 To). Cette machine héberge le juge local (Ollama) et l'estimateur de complexité ; les modèles cibles sont appelés à distance. Les latences d'inférence locale mesurées sont donc indicatives de ce poste, non d'un serveur de production (cf. §4.2).

A.1.0.0.2 Modèles cibles (via OpenRouter). Pool final : léger `qwen/qwen-2.5-72b-instruct`, lourd `anthropic/claude-opus-4.8`. Modèles écartés au cours de la sélection du léger : `meta-llama/llama-3.2-3b-instruct`, `openai/gpt-4o-mini`, `qwen2.5:7b/14b`, `deepseek/deepseek-v3.2`, `google/gemini-2.5-flash` et `-flash-lite`, `qwen/qwen3.5-flash` (cf. §5.6). Le modèle générateur des checklists RocketEval est `claude-sonnet-4.6`, distinct du léger et du lourd évalués (neutralité). Paramètres d'appel : température 0, plafond de génération `max_tokens = 4096`, trois relances sur erreur transitoire avec backoff exponentiel.

A.1.0.0.3 Juge local (via Ollama). `gemma2:9b` pour le juge de référence (100% en discrimination grossière) ; `gemma2:2b` conservé pour les comparaisons. Notation RocketEval, un appel local par critère de checklist, agrégation $q = \#oui/\#critères$.

A.1.0.0.4 Estimateur de complexité. `scikit-learn` RandomForest (200 arbres) sur les attributs de fond (n , p , $|\delta|$), plus les embeddings `sentence-transformers` (`all-MiniLM-L6-v2`) pour la variante combinée. Modèle persisté chargé en mémoire, inférence à un seul processus : 3,4 ms par intent (cf. §4.2).

A.1.0.0.5 Reproductibilité. Tirages aléatoires fixés (graine 42) pour l'échantillonnage équilibré et la stratégie RANDOM. Chaque chiffre empirique du mémoire provient d'un run daté et rejouable du

dépôt (méthodologie code-first).

A.2 Mise en œuvre et rejeu

A.2.0.0.1 Interface en ligne de commande. Le système s'exécute sur un intent unique par `python -m app.cli "<intent>"`, qui affiche chaque étage de la chaîne : attributs extraits et classe de complexité prédite, tableau des candidats du pool avec leur qualité attendue, leur coût et leur latence, modèle retenu et motif de la décision, réponse obtenue, puis notation du juge critère par critère. L'option `--stage decision` s'arrête avant l'appel au modèle : la décision de routage est alors instantanée et gratuite, ce qui permet d'explorer la politique sans consommer de budget. La chaîne complète s'exécute en une vingtaine de secondes, dominée par la génération.

Le cadrage envoyé aux modèles cibles est partagé entre cette interface et les campagnes de mesure (`app.llm.prompting`), de sorte qu'une démonstration interroge les modèles dans les conditions exactes des chiffres publiés.

A.2.0.0.2 Conteneurisation. Deux images sont définies. L'image `runtime` porte le routeur, le juge et les tests, sans les dépendances d'entraînement. L'image `full` y ajoute de quoi réentraîner l'estimateur, notamment la variante à embeddings de phrase. Des services de composition couvrent les usages courants : exécution d'un intent, suite de tests, campagne de mesure, réentraînement.

A.2.0.0.3 Artefacts versionnés. Le dépôt porte l'estimateur de complexité persisté ainsi que les résultats de chaque campagne, si bien qu'un clone permet de router un intent et de recalculer les tableaux de ce mémoire sans refaire un seul appel payant. Cette condition n'est pas cosmétique : sans le modèle persisté, aucun routage n'est possible, et sans les résultats, aucune agrégation n'est vérifiable.

A.3 Exemples d'intents et réponses

A.3.1 Un intent annoté du jeu de données

L'intent suivant, issu de l'amorce écrite à la main, illustre le schéma d'annotation décrit en §5.1.1 (attributs de la table 5.1) : une lecture d'état directe, un seul domaine, une à deux entités, donc un niveau simple selon la grille de la table 5.2.

```
id: ran-read-throughput
text: "What is the current downlink throughput on cell gNB-042?"
domain: ran
expected_complexity: simple
criticality: low
slice_type: null
```

A.3.2 Checklist RocketEval générée pour cet intent

La checklist ci-dessous a été générée par `claude-sonnet-4.6` pour l'intent précédent, selon la méthode RocketEval décrite en §5.3.1. Elle compte sept critères binaires, propres à cet intent ; le juge local répond oui ou non à chacun, et le score est la proportion de critères satisfaits. Les critères sont reproduits tels quels (la chaîne d'évaluation opère en anglais).

1. "Does the response avoid fabricating or inventing a specific numeric downlink throughput value for gNB-042 that it cannot know without live network access?"
2. "Does the response explain at least one concrete method (e.g., CLI command, SNMP OID, O1/NETCONF query, vendor-specific NMS tool, or KPI counter) that an operator can use to retrieve the current downlink throughput for gNB-042?"
3. "Does the response reference technically correct RAN-domain metrics or counters relevant to gNB-042 downlink throughput (e.g., DRB.UETpDl, PDCP layer throughput, PRB utilization, or equivalent 5G NR KPIs)?"
4. "Does the response clearly acknowledge that it has no live access to the network and therefore cannot provide the current real-time value?"
5. "Does the response avoid presenting any placeholder or example throughput figure as if it were the actual current value for gNB-042?"
6. "Does the response correctly distinguish between downlink-specific metrics and uplink or aggregate metrics when describing how to obtain the data?"
7. "Does the response provide actionable guidance that is specific enough for a network operator to follow without requiring significant additional research?"

Les deux premiers critères relèvent de la couverture obligatoire imposée au générateur de checklists (détection de la fabrication de données, explication du moyen d'obtenir la donnée) ; le troisième porte la correction technique propre au domaine RAN. C'est ce type d'item vérifiable qui corrige l'aveuglement à l'hallucination constaté avec la checklist fixe (§5.3.1).

A.4 Métriques classiques d'évaluation de texte

Le corps du mémoire (§2.4.1) écarte les métriques lexicales au profit du paradigme LLM-as-a-Judge. Leur définition est rappelée ici.

BLEU (*Bilingual Evaluation Understudy*, Papineni *et al.*, 2002) calcule la précision modifiée des n-grammes communs entre une sortie candidate et un ensemble de références, pondérée par une pénalité de brièveté. Sa faiblesse est lexicale : deux paraphrases équivalentes aux lexiques disjoints obtiennent un score bas.

ROUGE (*Recall-Oriented Understudy for Gisting Evaluation*, Lin, 2004) évalue le rappel des n-grammes ou des plus longues sous-séquences communes présents dans la référence. Elle partage avec BLEU la dépendance à une référence textuelle fixe.

A.4. MÉTRIQUES CLASSIQUES D'ÉVALUATION DE TEXTE

BERTSCORE [41] apparie chaque *token* de la sortie à son plus proche voisin dans la référence par similarité cosinus d'embeddings contextuels, puis agrège en précision, rappel et F_1 . Évaluée sur 363 systèmes, elle améliore la corrélation avec l'humain face à BLEU et ROUGE, mais reste tributaire d'une référence et capture mal la cohérence d'un texte long.